

SHERLOC: Structured Diagnostic Localization for Code Repair Agents

Hovhannes Tamoyan^{1,2}, Sean Narenthiran¹, Erik Arakelyan¹, Mira Mezini^{2,3} and Boris Ginsburg¹

¹NVIDIA, Santa Clara, CA 95051, USA, ²TU Darmstadt, Darmstadt, Germany, ³hessian.AI & National Research Center for Applied Cybersecurity ATHENE

LLM agents solve repository-level coding tasks through multi-turn tool use, but utilize half their budget on locating faults before editing. Dedicated localization frameworks have emerged, yet are still evaluated as file retrieval rather than actionable diagnosis, producing locations without the diagnostic context a repair agent needs. We introduce SHERLOC (*Structured Hypothesis-driven Exploration and Reasoning for Localization*), a training-free framework pairing a reasoning LLM with compact repository tools and self-recovery, without fine-tuning or multi-agent orchestration. SHERLOC reaches state-of-the-art localization across model scales: 84.33% accuracy@1 on SWE-BENCH LITE and 81.27% recall@1 on SWE-BENCH VERIFIED; at $\sim 30\text{B}$ parameters, it matches or outperforms other agentic methods. Injecting our locations and diagnostic findings into repair agents yields an average +5.95 pp resolve-rate gain from the best SHERLOC result per setting on SWE-BENCH VERIFIED. SHERLOC cuts localization and total tokens by 36.7% and 23.1% on average.¹

1. Introduction

Repository-level code repair increasingly relies on LLM agents that interleave extended reasoning with structured tool calls over a codebase [Wei et al., 2022, Yao et al., 2023, Schick et al., 2023]. A canonical benchmark for evaluating these agents is SWE-BENCH [Jimenez et al., 2024], built from GitHub issues and their resolving pull requests: from a bug report and a repository snapshot, the agent must propose and verify a bug-fixing code patch. Yet how agents allocate their interaction budget within this repair loop remains underexplored. In a systematic study spanning 5 LLMs and 2 agent frameworks, we find that agents spend on average 18.5 turns (48% of total interaction) locating faults before their first patch, consuming over 320k tokens per instance (Section L). This makes localization both a performance bottleneck and a dominant compute cost, consuming context and interaction budget that could otherwise support patch construction.

Most localization methods retrieve faulty files and functions [Zhou et al., 2012, Gangi Reddy et al., 2026, Yu et al., 2025]; some, e.g., OrcaLoca and SWE-Debate [Li et al., 2026], also produce free-form report text. This view is necessary but incomplete: a file path tells the agent where to look, but not why, and a file path without a root-cause hypothesis leaves downstream repair under-specified. We hypothesize that explicitly producing candidate locations, root-cause analysis, and actionable solution guidance yields a richer, more transferable localization, and ask: **(RQ1)**

¹<https://github.com/NVIDIA-NeMo/Skills/tree/main/recipes/sherloc>

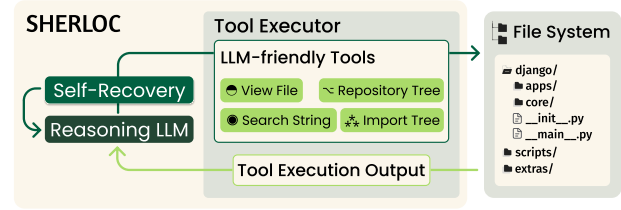


Figure 1: **Overview of SHERLOC.** A reasoning LLM interacts with a tool executor over four LLM-friendly tools, with a self-recovery layer correcting failures. SHERLOC achieves state-of-the-art file-level localization on SWE-BENCH LITE and VERIFIED. In downstream code repair, its best result per setting averages a 5.95 pp resolve-rate gain; On average SHERLOC cuts localization and total tokens by 36.7% and 23.1%.

Can a single reasoning LLM with a compact, structured tool interface match task-specifically trained localizers without fine-tuning or multi-agent orchestration? **(RQ2)** When do its locations and findings transfer to code-repair agents, and how does the resolve rate and token cost shift?

We introduce SHERLOC (*Structured Hypothesis-driven Exploration and Reasoning for Localization*), a training-free framework coupling a reasoning LLM with a compact suite of LLM-friendly repository tools (file viewing, code search, repository-tree inspection, and import-graph navigation; Figure 1) and lightweight self-recovery mechanisms (context trunca-

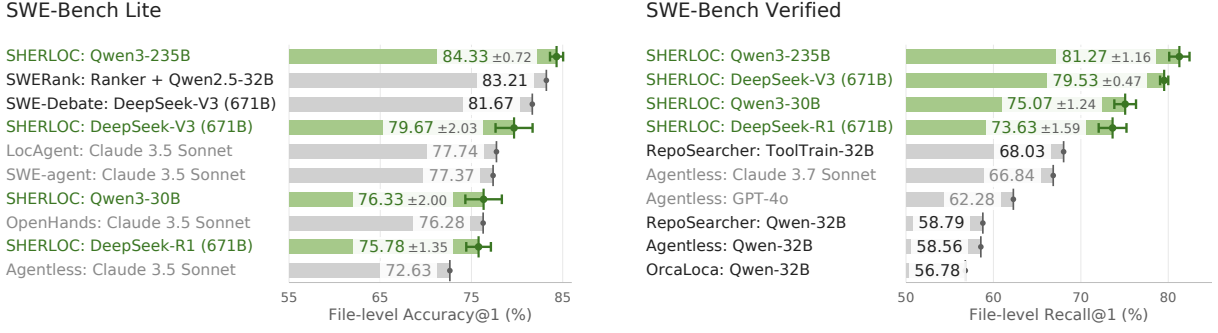


Figure 2: **Localization performance across benchmarks.** Left: file-level accuracy@1 on SWE-BENCH LITE; right: file-level recall@1 on SWE-BENCH VERIFIED. Labels use compact model names for readability; proprietary model labels appear in light gray. SHERLOC results are means over 3 seeds ($\pm$ std); prior systems report single runs. Backbone sizes differ across systems; Section 4.4 provides a controlled cross-model analysis.

tion, loop detection, malformed-tool-call repair, final-turn synthesis). For each predicted location, SHERLOC emits a structured diagnostic *finding*.

For (RQ1), we evaluate SHERLOC across model families on SWE-BENCH LITE and SWE-BENCH VERIFIED. It reaches state-of-the-art file-level localization on both benchmarks and remains competitive at a matched ~ 30 B scale, without supervised fine-tuning, reinforcement learning, or multi-agent debate. Ablations show that the tool suite, self-recovery mechanisms, and reasoning mode all contribute, while implicit-knowledge controls show that the gains persist when file paths in the issue are masked.

Strong localization does not automatically translate to better repository-level issue resolution. For (RQ2), we inject our locations and findings into two code-repair frameworks, OpenHands [Wang et al., 2025] and SWE-Agent [Yang et al., 2024], across 5 repair-agent backbones. Across the 10 model-framework pairs, the per-pair best of unfiltered SHERLOC and quality-filtered SHERLOC (QF SHERLOC) averages a +5.95 pp resolve-rate gain; deployable, unfiltered SHERLOC averages +4.39 pp, with 36.7% and 23.1% fewer localization and total tokens.

SHERLOC improves or preserves the resolve-rate point estimate in eight of ten settings and reduces localization tokens in all ten (Figures 3 and 4). QF SHERLOC mitigates the only observed case of negative transfer. Thus, transfer is positive on average but quality-mediated.

Our main contributions are:

- **A training-free, tool-augmented localization framework.** SHERLOC pairs a single reasoning LLM with a compact suite of LLM-friendly repository tools and lightweight self-

recovery, without task-specific training or multi-agent orchestration, and reaches state-of-the-art file-level localization on SWE-BENCH LITE (84.33% accuracy@1) and SWE-BENCH VERIFIED (81.27% recall@1).

- **Diagnostic findings as structured localization context.** Alongside candidate locations, SHERLOC produces structured findings with 5 fields (location explanation, root cause, solution idea, dependencies, testing impact) for each predicted location.
- **A structure-agnostic localization metric.** We propose chunk-level metrics over (start, end) line-number spans (coverage recall, precision, and average tightness) as a structure-agnostic alternative to function-, class-, or module-level evaluation, since not every patch target sits within a syntactic unit.
- **Cross-family transfer with validity controls.** SHERLOC transfers across model families (79.53% recall@1 with DeepSeek-V3-0324); implicit-knowledge controls show its gains persist when issue paths are masked (79.96% vs. 81.27%), isolating active exploration from parametric familiarity with public SWE-BENCH repositories.
- **Dual-axis transfer to downstream code repair.** Injecting SHERLOC’s findings into existing code-repair agents yields an average +5.95 pp resolve-rate gain from the best SHERLOC result per setting; SHERLOC cuts localization and total tokens by 36.7% and 23.1% across five repair backbones and two frameworks; a retrospective QF SHERLOC analysis shows that excluding unreliable findings can mitigate negative transfer.

Repair model / framework	Qwen3-30B / SA	44.7	40.2	54.0	52.2	76.0
	Qwen3-30B / OH	49.4	40.2	53.0	52.4	73.4
	Coder-Next / SA	45.2	43.2	53.2	52.0	64.6
	Coder-Next / OH	44.6	39.2	53.2	54.6	68.4
	Next-80B / SA	38.8	33.8	49.4	50.6	71.0
	Next-80B / OH	38.6	39.2	50.4	47.8	68.4
	MiniMax / SA	74.4	61.0	70.4	76.6	89.2
	MiniMax / OH	72.2	64.0	67.2	72.8	87.8
	Qwen3-480B / SA	63.0	58.6	63.0	64.0	86.2
	Qwen3-480B / OH	61.6	57.8	62.6	62.8	82.6
		Baseline	Shuffled	SHERLOC	QF SHERLOC	Oracle
		Intervention				

Figure 3: **Downstream resolve rates of LLM repair agents on SWE-Bench Verified.** Each cell reports resolve rate in percentages; within each row, non-Oracle conditions are shaded blue from light (worst) to dark (best). In row labels, SA denotes SWE-Agent and OH denotes OpenHands. Oracle GPT-5.2 (light gray) is an upper reference, not a deployable intervention. Bold values indicate the best deployable result in each row, comparing Baseline with SHERLOC. Paired differences and 95% confidence intervals appear in Table 14.

2. Related Work

Tool-augmented LLM reasoning. SHERLOC builds on reasoning-augmented tool use [Wei et al., 2022, Yao et al., 2023, Schick et al., 2023] and on test-time scaling that adapts reasoning compute to problem difficulty [Wu et al., 2025, Zhao et al., 2025]. Unlike prior localization systems built on supervised fine-tuning, reinforcement learning, or multi-agent orchestration, ours uses a single LLM with a fixed, structured tool set.

LLM-based code localization. Recent LLM-based methods explore repositories interactively or reason over code graphs: SWE-Debate adjudicates fault-propagation traces through multi-agent debate [Li et al., 2026]; OrcaLoca uses priority-based action scheduling and distance-aware context pruning [Yu et al., 2025]; CoSIL and LocAgent guide LLM search through code-graph structure [Jiang et al., 2025, Chen et al., 2025]; RepoSearcher/ToolTrain uses tool-integrated reinforcement learning [Ma et al., 2026]. SHERLOC differs in design: no training, no auxiliary structures, a compact set of LLM-friendly

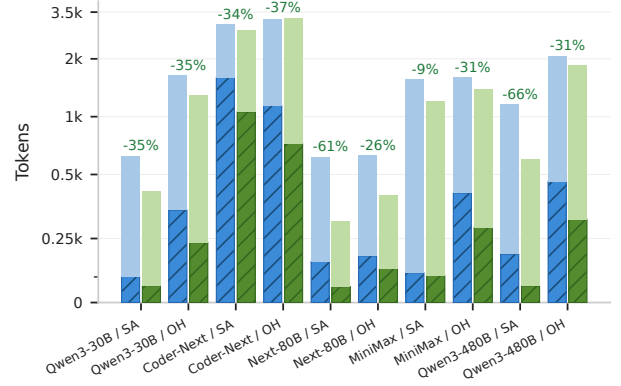


Figure 4: **Search-efficiency gains from SHERLOC findings.** Localization-phase (hatched) and full-run (solid) per-instance token costs: baseline (blue) vs. SHERLOC (green); the delta above each pair is the localization-token change relative to baseline. In x-axis labels, SA denotes SWE-Agent and OH denotes OpenHands. The largest reductions in localization-token usage occur with bigger models, even when their resolve rates are already saturated. Full numbers in Section L.

repository actions, and self-recovery mechanisms that stabilize extended-thinking tool use. Whereas prior work typically targets file- or function-level entity locations, sometimes with prescriptive modification plans for downstream patching [Li et al., 2026], our method additionally produces structured *diagnostic* findings alongside each location.

Issue-based retrieval and ranking. Information-retrieval localization treats the bug report as a query and source files as documents. BugLocator [Zhou et al., 2012] ranks files using textual similarity and prior bug reports; BLUIR [Saha et al., 2013] incorporates structured source-code information; AmaLgam [Wang and Lo, 2014] combines version history, similar reports, and structure. Recent neural rankers such as SWERank learn issue-code similarity with task-specific training [Gangi Reddy et al., 2026]. These methods output ranked locations rather than diagnostic reports; SHERLOC instead frames localization as active repository reasoning yielding structured findings.

Fault localization with execution signals. Classical fault localization assumes access to executable tests or dynamic traces: **spectrum-based** methods rank by suspiciousness from coverage and pass/fail statistics [Jones et al., 2002, Abreu et al., 2007]; **mutation-based** methods inject synthetic faults and observe test-outcome changes [Moon et al., 2014, Pa-

padakis and Le Traon, 2015]; learning-based extensions train neural models over coverage or graph features [Li et al., 2019, Lou et al., 2021]; AutoFL uses LLM tool calls but still requires at least one failing test [Kang et al., 2024]. Unlike these execution-dependent methods, SHERLOC operates from the issue description and repository snapshot alone, without executing tests or observing coverage.

Localization granularity. Existing localization studies report at module, file, class, or function level [Kang et al., 2024, Chen et al., 2025]. For repository-level repair, this granularity is not always aligned with the target edit: patches may modify class-level declarations, imports, configuration logic, or coordinated regions across functions. Several recent function-level evaluations sidestep this mismatch by discarding 26 of the 300 SWE-BENCH LITE instances that lack an existing function or class target [Suresh et al., 2025, Chen et al., 2025, Gangi Reddy et al., 2026]. We therefore complement file-level metrics with structure-agnostic chunk-level metrics over line ranges (coverage recall, precision, and average tightness) that represent predictions and ground truth as (start, end) line-number spans, making no assumption about which syntactic unit contains the patch target while retaining all benchmark instances.

Issue-resolution agents and benchmark validity. Issue-resolution systems target patch correctness directly: Agentless [Xia et al., 2025] and AutoCodeRover [Zhang et al., 2024] decompose the task into localization, patch generation, and validation; agentic systems such as SWE-Agent and OpenHands integrate navigation, editing, and execution in a single loop [Yang et al., 2024, Wang et al., 2025]. We instead evaluate localization directly and, in separate experiments, inject our findings into existing repair agents to test downstream transfer. Because high file-identification accuracy on SWE-BENCH can partly reflect pretraining familiarity [Liang et al., 2026], we apply implicit-knowledge, shuffled, and masked controls to separate benchmark-specific signal from tool-assisted diagnostic reasoning.

3. Method

We formulate localization as an iterative, tool-mediated reasoning problem: given a natural-language issue report and a repository snapshot, the system must identify code regions likely to require modification and produce a diagnostic finding, not merely a ranked location, that explains the suspected root cause and solution direction. SHERLOC im-

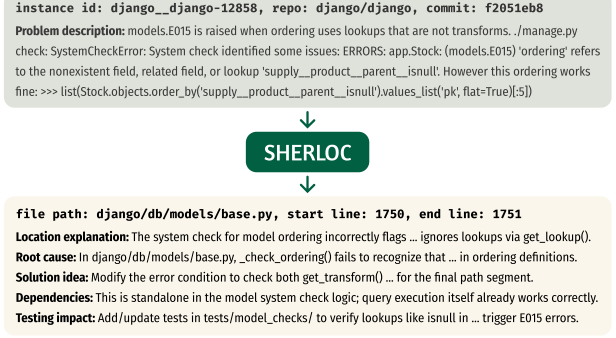


Figure 5: **Example SHERLOC input and output.** Given a SWE-BENCH VERIFIED Django issue, SHERLOC predicts a line span in `django/db/models/base.py` and emits a structured diagnostic finding with the location explanation, root cause, solution idea, dependencies, and testing impact.

plements this with four components (Figure 1): a reasoning LLM, a deterministic executor mediating repository access, a compact suite of LLM-friendly tools, and lightweight self-recovery. The model alternates between reasoning and structured actions until it has enough evidence to emit final findings and locations. Its hypothesis state is the accumulated conversation history, including prior observations; no external memory store is maintained.

Design principle: reasoning-native tool use. Rather than synthesizing arbitrary shell commands or Python scripts for repository actions, the model selects from a small fixed set of tools, each with a structured schema for execution (e.g., file path and optional line range) and bounded, model-readable observations such as snippets, paths, line ranges, and dependency summaries. This keeps repository exploration legible while preserving a deterministic execution boundary: the model chooses the action, but the executor validates and runs it, avoiding incidental command construction or shell-debug failure modes that prior work has shown to derail agent trajectories under raw shell access [Yang et al., 2024]. It also lets us use reasoning models, which excel at multi-step deliberation but are not necessarily trained for external tool APIs, without task-specific tool-use fine-tuning.

3.1. Initialization and Context

Before prompting, we construct a filtered repository view that removes directories unlikely to contain production logic, such as documentation, build artifacts, dependency folders, and version-control metadata (e.g., `/docs`, `/.git`). The initial prompt combines

this filtered tree with the issue description, tool descriptions, and required output format, giving the model a global map of the project without loading source files.

3.2. Tool Suite

The suite is limited by design: enough to expose relevant repository evidence, but not so broad as to invite open-ended software-engineering actions. **View File** inspects a file, optionally restricted to a line range, with fuzzy path matching that suggests corrections for near-miss names. **Codebase Search** performs repository-wide literal search and returns matching snippets with surrounding context. **Repository Tree** displays the filtered file hierarchy (like shell `TREE`), letting the model regain global context. **Connected Tree** summarizes import relationships (direct and reverse) to follow dependencies across modules.

3.3. Iterative Interaction Loop

At the core of SHERLOC is a bounded interaction loop of at most 20 turns; at each turn the model either makes a tool call or terminates with final locations and findings. The loop has four steps. **Reasoning and action selection**: the LLM receives the issue, repository context, and previous observations, then decides whether additional evidence is needed and, if so, emits one structured action. **Action parsing**: a parser extracts a tool call or a final answer; final answers contain a `findings` block (5 fields: location explanation, root cause, solution idea, dependencies, testing impact; see Figure 5 and Section J.1) and a `locations` block. **Tool execution**: the executor runs the action and appends the observation to the conversation history. **Termination**: the loop ends when a valid final answer is parsed or when the step budget is exhausted, in which case a final-turn prompt forces synthesis from gathered evidence.

3.4. Self-Recovery Mechanisms

We add recovery for common failure modes in multi-turn tool use (Section H): context management (truncating older observations), loop detection (warnings for repeated unproductive calls), implicit tool-call recovery (parsing malformed but unambiguous requests), response-length management (re-prompting when generation approaches safe limits), and final-turn prompting (forcing synthesis when the step budget is exhausted).

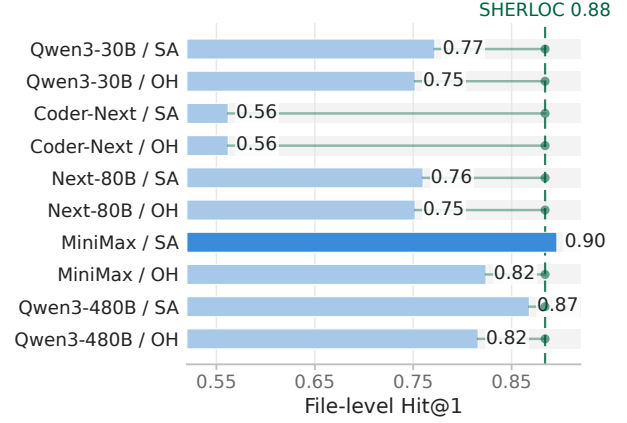


Figure 6: **Localization headroom by downstream LLM agent.** File-level Hit@1 of each code repair LLM agent’s own localization (blue) vs. SHERLOC (green dashed line, 0.88). The gap between each bar and the SHERLOC line is the agent’s headroom; weaker agents have larger headroom and gain more from external findings. Full numbers in Section L.

4. Results

We evaluate SHERLOC along four axes: (i) the quality of its predicted code locations and (ii) the robustness of those predictions to benchmark contamination and to LLM backbone choice (addressing **RQ1**; Sections 4.1, 4.3 and 4.4); and (iii) the transfer of its outputs to downstream code repair, gated by (iv) the quality of its diagnostic findings under an LLM judge (addressing **RQ2**; Section 4.6). We report file-level metrics (precision, recall, F1, exact match, hit@1) over the predicted file set and structure-agnostic chunk-level metrics (coverage recall, precision, average tightness) over (start, end) line-number spans; definitions in Section B.

4.1. Localization Performance and Efficiency

We evaluate our framework across SWE-BENCH LITE [Jimenez et al., 2024] and SWE-BENCH VERIFIED [Chowdhury et al., 2024] using 4 backbone models and 3 seeds. Figure 2 compares against leading localizers including SWE-Debate [Li et al., 2026], SWERank [Gangi Reddy et al., 2026], LocAgent [Chen et al., 2025], SWE-agent [Yang et al., 2024], OpenHands [Wang et al., 2025], and Agentless [Xia et al., 2025]. Following prior localization work [Li et al., 2026, Gangi Reddy et al., 2026, Chen et al., 2025], we report file-level accuracy@1 as the fraction of instances whose top-ranked predicted file is a gold file and recall@1 as the mean fraction of gold files recovered by the top-ranked prediction.

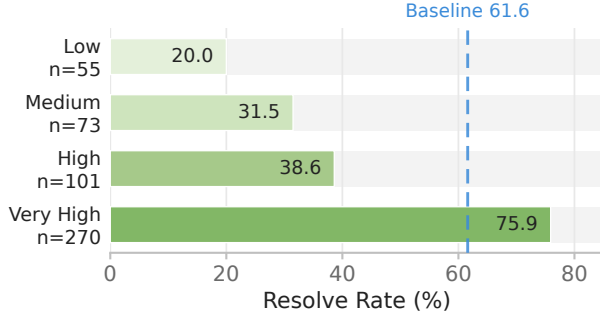


Figure 7: **Resolve rate by composite finding-quality bucket.** SWE-BENCH VERIFIED / OPENHANDS / Qwen3-Coder-480B-A35B-Instruct. Buckets: Low [1.0,2.0], Medium (2.0,3.0], High (3.0,4.0], Very High (4.0,5.0]. Very-high-quality findings resolve at 75.9%; low-quality findings actively underperform the 61.6% baseline.

On SWE-BENCH LITE, our framework with Qwen3-235B-A22B-Thinking-2507 (Qwen3-235B) [Yang et al., 2025a, Qwen Team, 2025a] achieves $84.33 \pm 0.72\%$ accuracy@1 in 5.0 turns on average; the strongest plotted prior result is the fine-tuned SWERank retrieve-and-rerank pipeline (83.21%). At the chunk level, it reaches 39.14% coverage recall and 44.54% precision.

On SWE-BENCH VERIFIED, the best configuration (Qwen3-235B) achieves $81.27 \pm 1.16\%$ recall@1 with 4.7 average turns, a 13.2 pp improvement over the previous state of the art, RepoSearcher / ToolTrain-32B [68.03%; Ma et al., 2026]. At a matched ~ 30 B scale and without task-specific training, Qwen3-30B-A3B-Thinking-2507 reaches $75.07 \pm 1.24\%$ recall@1, outperforming ToolTrain-32B by 7.0 pp. At the chunk level, it reaches 41.28% coverage recall and 53.47% precision. Because prior systems do not report chunk-level metrics, these span scores primarily characterize how tightly SHERLOC’s predicted regions cover the gold edit spans rather than serving as a direct baseline comparison.

DeepSeek-V3-0324 [DeepSeek-AI et al., 2024, DeepSeek-AI, 2025b] reaches $79.53 \pm 0.47\%$ on Verified, confirming cross-family transfer. Full per-backbone numbers (including chunk-level metrics and tool-engagement) appear in Table 3; the complete metric grid is in Section B.

At the trajectory level, the search remains compact while scaling with problem difficulty: turns rise from 4.22 (easy) to 5.05/5.21 (medium/hard) and total tokens from 21.3k to 32.8k/35.1k (Sections F and G).

4.2. Component Ablations

To keep the full component grid tractable, we ablate each component of our framework and show the impact of each separate tool and mechanism on a fixed sample of 100 instances drawn uniformly from the SWE-Gym development set [Pan et al., 2025] (full breakdown in Section A). Removing any single tool or self-recovery mechanism degrades localization, with View File (-7.0 pp F1) and the final-turn prompt (-5.0 pp F1) as the largest contributors, indicating that both code inspection and synthesis pressure are central.

4.3. Implicit Knowledge vs. Tool-Assisted Reasoning

Because SWE-BENCH repositories are public and widely represented in pretraining corpora, localization accuracy can reflect both active reasoning and implicit knowledge of familiar codebases [Liang et al., 2026]. We treat this as a validity concern and conduct a controlled masking study with Qwen3-235B on SWE-BENCH VERIFIED, progressively removing repository access and identifying information (Table 5). Even when tools, the repository tree, and all file and module paths in the issue are masked, the model still achieves 57.86% recall@1 from the issue text alone, often inferring the correct file from error messages, API names, and domain conventions. We interpret this primarily as repository familiarity with widely used foundational libraries, not as direct evidence that individual benchmark instances are memorized. Manually inspecting a random sample of 50 successful instances from this setting confirms 2 patterns: reliance on (i) well-known APIs and libraries, and (ii) distinctive error messages that uniquely identify a module. Quantitatively (Table 6), implicit recall concentrates in popular projects (85.3% on scikit-learn, 87.5% on requests), while less common ones like pylint and seaborn fall to 33.3%.

Crucially, this baseline familiarity does not explain SHERLOC’s full performance. When tools and the repository tree are retained but explicit paths are removed from the issue text, SHERLOC reaches 79.96% recall@1, only 1.3 pp below the unmasked full system (81.27%). The ≈ 22 pp gap between this masked-issue setting and the heavily-masked text-only setting estimates the contribution of active repository exploration beyond parametric familiarity.

4.4. Backbone Generalization and Tool Engagement

To separate the SHERLOC method from the backbone, we evaluate 5 models from 2 fami-

lies: Qwen3-235B-A22B-Thinking-2507, Qwen3-30B-A3B-Thinking-2507, and Qwen3-30B-A3B-Instruct-2507 [Yang et al., 2025a, Qwen Team, 2025a,b,c]; DeepSeek-V3-0324 [DeepSeek-AI et al., 2024, DeepSeek-AI, 2025b]; and DeepSeek-R1-0528 [Guo et al., 2025, DeepSeek-AI, 2025a]. Runs use identical code, prompts, tools, and self-recovery; only the backbone changes. Each uses 3 seeds on SWE-BENCH VERIFIED; recall, mean turns, and zero-tool rate are in Table 3.

First, **SHERLOC transfers across model families**: DeepSeek-V3-0324 reaches 79.53% recall, only 1.7 pp below the Qwen3-235B ceiling and 11.5 pp above the best published baseline [ToolTrain-32B, 68.03%; Ma et al., 2026]. Second, **tool engagement matters**: DeepSeek-R1-0528 makes 41% fewer tool calls and 32% fewer turns than Qwen3-235B, with 10% producing locations without any tool call; those zero-tool instances reach 69.3% recall vs. 73.6% on its tool-using subset, so active exploration helps even strong reasoning models. Third, **scale alone is not enough**: Qwen3-30B outperforms the larger R1-0528 (75.07% vs. 73.63%) while taking the most turns (7.2 on average, vs. 4.7 for Qwen3-235B and 3.2 for R1-0528). R1-0528’s under-exploration is consistent with DeepSeek’s lack of a marked system-role token: the prompt is delivered as bare preamble, reducing tool-use compliance. The Qwen3-30B-Thinking vs. Qwen3-30B-Instruct ablation (Section A.1) reinforces this: reasoning is critical for sustaining the multi-turn protocol.

4.5. LLM-as-Judge Scoring of Findings

How reliable are the diagnostic findings that SHERLOC produces? SHERLOC produced locations and findings for 499 of the 500 SWE-BENCH VERIFIED instances, with one instance yielding no prediction. We score these 499 findings with an LLM-as-judge. We use GPT-5.2 [OpenAI, 2025] as a ground-truth-patch-conditioned judge that rates each finding from 1 to 5 on three dimensions: **root-cause correctness**, **location accuracy**, and **solution actionability** (prompt details in Section H; full quality-scoring breakdown in Section J). The composite quality score is the mean of these three dimensions, and we call a finding *high-quality* when this composite is ≥ 4.0 . The scores are right-skewed: 270 (54%) fall in Very High (> 4.0), 101 (20%) in High (3.0 to 4.0), 73 (15%) in Medium (2.0 to 3.0), and 55 (11%) in Low (≤ 2.0) (Table 7). The ≥ 4.0 threshold therefore retains 317/499 scored findings (63.5%).

4.6. Downstream Transfer to Code Repair Task

Does better localization translate into better downstream issue resolution? We measure code-repair *resolve rate* (fraction of generated patches that pass the held-out tests) on all 500 SWE-BENCH VERIFIED instances [Chowdhury et al., 2024], using 2 agentic frameworks, SWE-Agent [Yang et al., 2024] and OpenHands [Wang et al., 2025], across 6 setups (Figure 3; full grid in Table 11): **Baseline** (vanilla agent execution); **Masked** (baseline with file paths and repository names masked, extending the implicit-knowledge control from Section 4.3); **Shuffled findings** (a SHERLOC location and finding from a different random instance); **SHERLOC findings** (SHERLOC’s predicted locations and findings); **QF SHERLOC** (retrospectively selects the SHERLOC outcome for scored findings with judge score ≥ 4.0 and the Baseline outcome below the threshold); and **Oracle GPT-5.2** (upper-bound finding conditioned on the ground-truth patch).

Transfer gains depend on model capability. SHERLOC findings improve or preserve the resolve-rate point estimate in eight of ten settings and reduce localization tokens in all ten (Figures 3 and 4). *Smaller and mid-sized models gain most from SHERLOC findings.* For Qwen3-Coder-30B-A3B [Qwen Team, 2025e], SWE-Agent rises from 44.7% to 54.0% and OpenHands from 49.4% to 53.0%; Qwen3-Coder-Next [Cao et al., 2026] and Qwen3-Next-80B-A3B-Instruct [Qwen Team, 2025g] show +8 to +12 pp improvements across both frameworks (Figure 3). These models have low baseline resolve and large localization headroom (Qwen3-Coder-Next reaches only 0.56 Hit@1 vs. our 0.88; Figure 6), so even moderate-quality findings close a real gap. *QF SHERLOC mitigates the only observed case of negative transfer.* For MiniMax-M2.5 [MiniMax, 2026], SHERLOC lowers resolve rate by 4 to 5 pp across the two frameworks (Figure 3). QF SHERLOC raises these results to 76.6% with SWE-Agent and 72.8% with OpenHands, but neither differs significantly from Baseline (Section K.2). *Token efficiency improves uniformly.* Across all 5 repair models and 2 frameworks, SHERLOC findings reduce localization tokens by 9 to 66% (36.7% on average) and total tokens by 23.1% on average (Figure 4 and Section L), giving the agent a targeted starting point and freeing context and interaction budget for repair. The largest cell-level efficiency gain is Qwen3-Coder-480B-A35B [Qwen Team, 2025d] / SWE-Agent, where injecting findings cuts localization tokens from 188.9k to 64.6k (−66%) while leaving resolve rate unchanged at 63.0%, showing that even for already-saturated repair agents the standalone localizer pays for itself in search cost alone.

The masked and shuffled controls remain close to baseline (Table 18), so gains come from instance-relevant diagnostic guidance, not surface leakage or unrelated findings.

4.7. Understanding Downstream Transfer

Finding quality is associated with downstream success. On the Qwen3-Coder-480B-A35B / OpenHands cell, very-high-quality findings resolve at 75.9%, compared with 20.0% for low-quality ones, and composite score correlates with repair outcome at $r=0.45$ (Figure 7). Using the ≥ 4.0 threshold defined in Section 4.5, we divide instances by the quality of their SHERLOC findings. Above the threshold, conditioning the repair agent on these findings raises resolve rate from 74.8% under Baseline to 80.4%; below the threshold, it lowers resolve rate from 38.5% to 31.3%. QF SHERLOC therefore retains the SHERLOC outcome only above the threshold and falls back to Baseline otherwise, mitigating negative transfer in Section 4.6. The pre-specified 4.0 threshold is the highest tested half-point cutoff retaining a majority of scored findings (317/499; 63.5%) and is within 2.1 pp of the best retrospective estimate at 4.5 (Table 10 and Fig. 8).

Patch-free verification provides a deployable confidence signal. Because QF SHERLOC relies on a ground-truth-patch-conditioned judge, it is retrospective. As an optional deployable safeguard for repair agents whose validation results indicate negative transfer, our patch-free verifier evaluates the canonical finding using the issue description and five auxiliary traces. Across 499 findings, its confidence score correlates with retrospective quality at $r=0.553$; at threshold 4.0, it selects 281 findings with 79.0% precision and 70.0% recall. This experiment measures agreement with retrospective quality labels, not downstream repair under verifier-based selection.

Solution content carries that signal. Two independent lenses converge: by per-dimension correlation on the Qwen3-Coder-480B-A35B / OpenHands cell, solution actionability has the strongest individual association with repair success ($r=0.46$, vs. $r=0.41$ for root-cause and $r=0.36$ for location accuracy); by causal field ablation on Qwen3-Coder-30B-A3B / SWE-Agent (Table 9), removing the *solution idea* field produces the largest single-field drop while all other reduced variants still beat the baseline. The transferable signal therefore comes from correct root-cause and solution content, not from merely seeing a SHERLOC-formatted hint.

Chunk coverage predicts downstream repair success. To separate chunk coverage from file identi-

fication, we restrict the analysis to the 371 instances for which SHERLOC identifies all and only the correct files, so every file metric is 100%. Among these, 144 instances have complete coverage of the gold edit chunks and 227 have incomplete coverage. Across the ten SHERLOC repair settings, the complete coverage group resolves at 82.5%, compared with 59.9% for the incomplete coverage group, a +22.6 pp difference. The gap is positive in every setting and ranges from +18.8 to +27.4 pp (Table 13). Thus, chunk coverage provides information about downstream repair success even when file identification is perfect. Chunk metrics complement rather than replace standard file metrics.

Findings help most on multi-file bugs. On the 71 Verified instances requiring 2 or more file changes, SHERLOC findings improve resolve rate by +5.6 pp (32.4% vs. 26.8%).

Finally, we separate accuracy from efficiency transfer by injecting findings from the weaker Qwen3-30B SHERLOC (Hit@1 0.786 vs. 0.884; Table 19). These weaker findings no longer improve Qwen3-Coder-30B-A3B but still improve Qwen3-Coder-Next (SWE-Agent 45.2%→49.2%; OpenHands 44.6%→45.4%; Table 12), and still reduce localization tokens by 16 to 32% (24.5% on average) and total tokens by 1 to 17% (7.5% on average) across all 4 cells (Table 16). Localizer quality thus governs accuracy transfer; even imperfect findings reduce search cost by anchoring initial exploration.

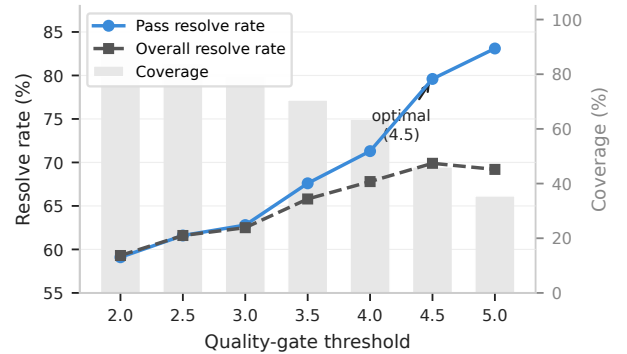


Figure 8: **Quality-threshold sweep.** Lines (left axis): Pass resolved (blue) is resolve rate among instances meeting the threshold; Overall resolved (dashed) is the filtered estimate with 61.6% baseline fallback for instances with rejected findings. Bars (right axis): coverage among the 499 scored findings. Stricter thresholds raise resolve rate while shrinking coverage; Overall resolved peaks at 4.5.

5. Conclusion

Our work on SHERLOC shows that structured diagnostic output, not just location retrieval, is the operative unit of useful localization. For **RQ1**, a single reasoning LLM with compact repository tools and self-recovery can reach state-of-the-art file-level localization without task-specific fine-tuning or multi-agent orchestration: 84.33% accuracy@1 on SWE-BENCH LITE and 81.27% recall@1 on SWE-BENCH VERIFIED, with matched-scale performance competitive with other agentic methods. For **RQ2**, injecting these locations and diagnostic findings into existing code-repair agents yields an average +5.95 pp resolve-rate gain from the best SHERLOC result per setting; On average SHERLOC cuts localization and total tokens by 36.7% and 23.1% across 5 repair backbones and 2 frameworks. A retrospective QF SHERLOC analysis shows that excluding unreliable findings can mitigate negative transfer.

We conclude that localization should be evaluated not just by file or function retrieval but by diagnostic actionability: **a correct file with a misleading diagnosis still misleads the downstream agent**. Beyond bug localization, structured findings may benefit repository-scale tasks such as test generation, regression triage, refactoring, and migration planning, where intermediate reasoning matters as much as the final action.

Limitations

Implicit-knowledge confound. Our implicit-knowledge controls show that $\approx 58\%$ of localization recall on SWE-BENCH VERIFIED is achievable from issue text alone, with strong per-repository concentration: scikit-learn and requests reach 85–88% masked recall while pylint and seaborn fall to 33% (Section E). Headline SWE-BENCH localization numbers, ours included, therefore partly reflect LLM pretraining familiarity with widely distributed open-source code rather than transferable code reasoning. Our masked-issue control (79.96% with tools retained) bounds but does not eliminate this confound; fully isolating pretraining familiarity would require evaluation on demonstrably unseen repositories.

Benchmark and framework scope. Code-repair transfer is measured on SWE-BENCH VERIFIED through two repair frameworks (SWE-AGENT, OPENHANDS) and five repair backbones. Generalization to private or demonstrably unseen repositories, other benchmarks such as SWE-BENCH MULTIMODAL [Yang et al., 2025b] and multilingual variants,

newer benchmark snapshots, additional agent frameworks, and non-Python repositories (JavaScript, Java, C++) remains open. The Connected Tree tool relies on a thin language-specific import parser that must be adapted for each supported language; the rest of SHERLOC is language-agnostic.

Compute cost of the headline configuration.

Our best numbers use Qwen3-235B-A22B-Thinking-2507 with up to 20 reasoning turns per instance, so the headline localization rows are not directly comparable on serving cost to the fine-tuned 32B baselines we cite. The matched-scale 30B row (75.07% on VERIFIED, +7.0 pp over ToolTrain-32B) is the most cost-comparable point of evidence for the method itself; the +6.2 pp from 235B over 30B partly reflects model scale.

Cross-model prompt sensitivity.

Identical prompts elicit different tool-use behavior across model families: DeepSeek-R1-0528, which does not support marked system roles, exhibits 10% zero-tool short-cutting under the same prompt text (Section 4.4). Systematic evaluation of commercial models such as GPT, Claude, and Gemini remains future work; they can be connected through a model/API adapter without changing the localization workflow, with the required integration points documented in the released code. Picking a backbone with system-prompt support sidesteps this; otherwise, reaching the reported numbers on a new backbone may require model-specific prompt adaptation rather than a strictly drop-in deployment.

Retrospective QF SHERLOC and patch-free verification.

The QF SHERLOC analysis uses GPT-5.2 [OpenAI, 2025] as an external judge that is shown the ground-truth patch. The judge-independent cells in Figure 3 (baseline, masked, shuffled, SHERLOC, Oracle GPT-5.2) provide non-judge evidence for the 2 transfer regimes, but the per-instance reliability analysis and the threshold sweep in Section J.3 depend on this external supervision. The QF SHERLOC row should therefore be read as an analysis-time selection experiment. Our patch-free verifier removes the ground-truth requirement, but we evaluate only its agreement with retrospective quality labels, not downstream repair under verifier-based selection.

Reproducibility Statement

SHERLOC is implemented using the NeMo-Skills framework [NVIDIA, 2024]. The exact prompts

(system prompt, tool descriptions, final-turn prompt, judge prompt) and inference parameters are provided in Section H. All open-weight models used in this work are publicly available. Localization uses Qwen3-235B-A22B-Thinking-2507 [Qwen Team, 2025a], Qwen3-30B-A3B-Thinking-2507 [Qwen Team, 2025c], Qwen3-30B-A3B-Instruct-2507 [Qwen Team, 2025b], DeepSeek-V3-0324 [DeepSeek-AI, 2025b], and DeepSeek-R1-0528 [DeepSeek-AI, 2025a]. Downstream repair uses Qwen3-Coder-30B-A3B [Qwen Team, 2025e], Qwen3-Coder-Next [Cao et al., 2026], Qwen3-Next-80B-A3B-Instruct [Qwen Team, 2025g], MiniMax-M2.5 [MiniMax, 2026], and Qwen3-Coder-480B-A35B [Qwen Team, 2025f]. The LLM-as-judge quality scorer, GPT-5.2 [OpenAI, 2025], is proprietary and accessed through its public API. Model names and versions are listed above, and the sampling parameters for every run are given in the *Sampling parameters* paragraph below. Evaluation is conducted on the public SWE-BENCH LITE and SWE-BENCH VERIFIED benchmarks using their official evaluation harnesses.

Computing infrastructure and budget. All experiments are inference-only and run on internal compute clusters of NVIDIA H100 (80 GB) and A100 (80 GB) GPUs. Backbones are served via SGLang [Zheng et al., 2024] for localization and vLLM [Kwon et al., 2023] for downstream code repair. Approximately 7,900 GPU-hours were used for the downstream repair experiments (5 repair backbones $\times$ 2 frameworks $\times$ 6 conditions), approximately 2,500 GPU-hours for the multi-backbone, multi-seed localization evaluations, and a few hundred GPU-hours for the component and reasoning-mode ablations and implicit-knowledge controls. The Qwen3-235B-A22B-Thinking-2507 localization runs used 16 GPUs for serving and averaged approximately 107 GPU-hours per 500 tasks, or 0.21 GPU-hours per task. Standard SHERLOC requires one localization run per issue; the optional patch-free verifier additionally requires five auxiliary localization runs and one verifier call.

Sampling parameters. Every SHERLOC localization run uses temperature = 0.99, top- p = 0.95, top- k = 0 (disabled), a per-call generation cap of 81,920 tokens, and at most 20 interaction turns. All downstream code-repair backbones use temperature = 0.7, top- p = 0.8, and top- k = 20 under the stock SWE-AGENT and OPENHANDS configurations. The LLM-as-judge quality scorer (GPT-5.2) uses temperature = 0.1 and a 300-token output cap. The prompts and remaining implementation details are in Section H.

Acknowledgments

We thank Somshubra Majumdar, Vahid Noroozi, Wasi Uddin Ahmad, Nikolai Ludwig, Mehrzad Samadi, Aleksander Ficek, and Siddhartha Jain (NVIDIA) for valuable discussions and feedback that helped shape this work. The work benefited from funding by the DFG (German Research Foundation) under the Excellence Strategy, EXC-3057.

References

- Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. On the accuracy of spectrum-based fault localization. In *Testing: Academic and Industrial Conference Practice and Research Techniques - MUTATION*, pages 89–98, 2007. doi: 10.1109/TAIC.PART.2007.13. URL <https://doi.org/10.1109/TAIC.PART.2007.13>.
- Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. Qwen3-Coder-Next technical report, 2026. URL <https://arxiv.org/abs/2603.00729>.
- Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL <https://aclanthology.org/2025.acl-long.426/>.
- Neil Chowdhury, James Aung, Jun Shern Chan, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench Verified. OpenAI Blog, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- DeepSeek-AI. DeepSeek-R1-0528 model card. Hugging Face, 2025a. URL <https://huggingface.co/deepseek-ai/DeepSeek-R1-0528>.

- DeepSeek-AI. DeepSeek-V3-0324 model card. Hugging Face, 2025b. URL <https://huggingface.co/deepseek-ai/DeepSeek-V3-0324>.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. DeepSeek-V3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1128. URL <https://aclanthology.org/P18-1128/>.
- Allen L. Edwards. Note on the “correction for continuity” in testing the significance of the difference between correlated proportions. *Psychometrika*, 13(3): 185–187, September 1948. doi: 10.1007/BF02289261. URL <https://doi.org/10.1007/BF02289261>.
- Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan-Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. SWERank: Software issue localization with code ranking. In *The Fourteenth International Conference on Learning Representations*, pages 78997–79011, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/7f6901ebab786e43b21530328fc989ca-Abstract-Conference.html.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang,

- Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. doi: 10.1038/s41586-025-09422-z. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue localization via LLM-driven iterative code graph searching. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*, pages 3034–3045, 2025. doi: 10.1109/ASE63991.2025.00249. URL <https://doi.org/10.1109/ASE63991.2025.00249>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, pages 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.
- James A. Jones, Mary Jean Harrold, and John Stasko. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering*, pages 467–477, 2002. doi: 10.1145/581339.581397. URL <https://doi.org/10.1145/581339.581397>.
- Sungmin Kang, Gabin An, and Shin Yoo. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering*, 1(FSE):1424–1446, 2024. doi: 10.1145/3660771. URL <https://doi.org/10.1145/3660771>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. SWE-Debate: Competitive multi-agent debate for software issue resolution. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, 2026. URL <https://conf.researchr.org/details/icse-2026/icse-2026-research-track/282/SWE-Debate-Competitive-Multi-Agent-Debate-for-Software-Issue-Resolution>.
- Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 169–180, 2019. doi: 10.1145/3293882.3330574. URL <https://doi.org/10.1145/3293882.3330574>.
- Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, pages 395–405, 2026. doi: 10.1145/3786583.3786882. URL <https://doi.org/10.1145/3786583.3786882>.
- Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 664–676, 2021. doi: 10.1145/3468264.3468580. URL <https://doi.org/10.1145/3468264.3468580>.
- Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. Enhancing issue localization agent with tool-interactive training. In

- Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, 2026. URL <https://conf.researchr.org/details/icse-2026/icse-2026-research-track/95/Enhancing-Issue-Localization-Agent-with-Tool-Interactive-Training>.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, June 1947. doi: 10.1007/BF02295996. URL <https://doi.org/10.1007/BF02295996>.
- MiniMax. MiniMax-M2.5 model card. Hugging Face, 2026. URL <https://huggingface.co/MiniMaxAI/MiniMax-M2.5>.
- Seokhyeon Moon, Yunho Kim, Moonzoo Kim, and Shin Yoo. Ask the mutants: Mutating faulty programs for fault localization. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*, pages 153–162, 2014. doi: 10.1109/ICST.2014.28. URL <https://doi.org/10.1109/ICST.2014.28>.
- NVIDIA. NeMo-Skills: A project to improve skills of large language models. GitHub repository, 2024. URL <https://github.com/NVIDIA-NeMo/Skills>.
- OpenAI. Update to GPT-5 system card: GPT-5.2. OpenAI System Card, December 2025. URL <https://openai.com/index/gpt-5-system-card-update-gpt-5-2/>.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47717–47737. PMLR, 2025. URL <https://proceedings.mlr.press/v267/pan25g.html>.
- Mike Papadakis and Yves Le Traon. Metallaxis-FL: Mutation-based fault localization. *Software Testing, Verification and Reliability*, 25(5–7):605–628, 2015. doi: 10.1002/stvr.1509. URL <https://doi.org/10.1002/stvr.1509>.
- Qwen Team. Qwen3-235B-A22B-Thinking-2507 model card. Hugging Face, 2025a. URL <https://huggingface.co/Qwen/Qwen3-235B-A22B-Thinking-2507>.
- Qwen Team. Qwen3-30B-A3B-Instruct-2507 model card. Hugging Face, 2025b. URL <https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507>.
- Qwen Team. Qwen3-30B-A3B-Thinking-2507 model card. Hugging Face, 2025c. URL <https://huggingface.co/Qwen/Qwen3-30B-A3B-Thinking-2507>.
- Qwen Team. Qwen3-Coder: Agentic coding in the world. Qwen Blog, July 2025d. URL <https://qwenlm.github.io/blog/qwen3-coder/>.
- Qwen Team. Qwen3-Coder-30B-A3B-Instruct model card. Hugging Face, 2025e. URL <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct>.
- Qwen Team. Qwen3-Coder-480B-A35B-Instruct model card. Hugging Face, 2025f. URL <https://huggingface.co/Qwen/Qwen3-Coder-480B-A35B-Instruct>.
- Qwen Team. Qwen3-Next-80B-A3B-Instruct model card. Hugging Face, 2025g. URL <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct>.
- Ripon K. Saha, Matthew Lease, Sarfraz Khurshid, and Dewayne E. Perry. Improving bug localization using structured information retrieval. In *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering*, pages 345–355, 2013. doi: 10.1109/ASE.2013.6693093. URL <https://doi.org/10.1109/ASE.2013.6693093>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- Tarun Suresh, Revanth Gangi Reddy, Yifei Xu, Zach Nussbaum, Andriy Mulyar, Brandon Duderstadt, and Heng Ji. CoRNStack: High-quality contrastive data for better code retrieval and reranking. In *The Thirteenth International Conference on Learning Representations*, pages 15800–15817, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/285b0e63256699c04dbf2c8d4163fba4-Abstract-Conference.html.
- Shaowei Wang and David Lo. Version history, similar report, and structure: Putting them together for improved bug localization. In *Proceedings of the 22nd International Conference on Program Comprehension*, pages 53–63, 2014. doi: 10.1145/2597008.2597148. URL <https://doi.org/10.1145/2597008.2597148>.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe

- Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, pages 65882–65919, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- Menghua Wu, Cai Zhou, Stephen Bates, and Tommi Jaakkola. Thought calibration: Efficient and confident test-time scaling. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14291–14305, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.722. URL <https://aclanthology.org/2025.emnlp-main.722/>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754. URL <https://doi.org/10.1145/3715754>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench Multimodal: Do AI systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, pages 2794–2829, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/07d6332ae36730707fdddb7a736d7b6c-Abstract-Conference.html.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. OrcaLoca: An LLM agent framework for software issue localization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 73416–73436. PMLR, 2025. URL <https://proceedings.mlr.press/v267/you25x.html>.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1592–1604, 2024. doi: 10.1145/3650212.3680384. URL <https://doi.org/10.1145/3650212.3680384>.
- Zhengyi Zhao, Shubo Zhang, Zezhong Wang, Huimin Wang, Yutian Zhao, Bin Liang, Yefeng Zheng, Binyang Li, Kam-Fai Wong, and Xian Wu. T²: An adaptive test-time scaling strategy for contextual question answering. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3731–3756, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.1

8653/v1/2025.emnlp-main.185. URL <https://aclanthology.org/2025.emnlp-main.185/>.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pages 62557–62583. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering*, pages 14–24, 2012. doi: 10.1109/ICSE.2012.6227210. URL <https://doi.org/10.1109/ICSE.2012.6227210>.

Part I: Localization Analyses

A. Component Ablations: Tools and Self-Recovery

Tables 1 and 2 evaluate the localization agent on the same 100 uniformly sampled SWE-Gym development issues. We ablate one component at a time while holding the base model, prompts, sampling setup, and all other tools or self-recovery mechanisms fixed.

The sample covers 10 of 11 repositories, with repository shares differing from the full development split by at most 4.3 pp. SWE-Gym follows the SWE-Bench instance-extraction and execution-based validation pipelines on separate Python repositories [Pan et al., 2025], providing a comparable development setting. We therefore interpret the ablation as evidence for qualitative component trends; exact effect sizes may vary on the full distribution.

File-level metrics (precision, recall, F1, exact match, and set accuracy) are computed over the predicted file set; chunk-level metrics (coverage recall and average tightness) are computed over (start, end) line-number spans. Parenthesized values are absolute-point deltas relative to the corresponding full-system row in the same table.

Across both ablations, the largest performance drops come from removing View File (tool suite) and the final-turn prompt (self-recovery), confirming that code inspection and forced final-turn synthesis are the two most load-bearing components.

A.1. Reasoning-Mode Ablation: Thinking vs. Instruct

To isolate the contribution of extended reasoning, we compare **Qwen3-30B-A3B-Thinking-2507** against its non-thinking counterpart **Qwen3-30B-A3B-Instruct-2507**, which shares the same parameter count, architecture, prompts, and tools. The Instruct model achieves only 10.2% recall on Verified and 11.7% on Lite, versus 74.0% and 76.7% for the Thinking model: a ~ 65 **pp gap**. The failure is not gradual degradation but near-total collapse: 87% of Instruct samples fail to produce any valid output (success rate 13%), with the model averaging 0.2 predicted locations per instance versus 1.3 for the Thinking variant. Without extended chain-of-thought, the model cannot sustain the multi-turn tool-use protocol that SHERLOC requires. This ablation, controlled within a single model family, shows that thinking-mode reasoning is critical for this multi-turn exploration and diagnosis protocol.

Table 1: **Tool-suite ablation on 100 SWE-Gym development issues.** Each row removes one localization tool while holding the base model, prompt, self-recovery settings, and evaluation set fixed. File-level columns report precision, recall, F1, exact match, and set accuracy; chunk-level columns report coverage recall and tightness. Parenthesized values are absolute-point deltas from the all-tools full system.

Ablated Tool	File-Level			File-Level (set)		Chunk-Level	
	Prec.	Rec.	F1	Exact	Acc.	Cov.Rec.	Tight.
All Tools	77.1	58.23	63.19	37.0	56.31	27.49	27.97
w/o View File	69.66 (-7.4)	51.70 (-6.5)	56.19 (-7.0)	35.0 (-2.0)	49.95 (-6.36)	22.32	19.40
w/o Codebase Search	71.73 (-5.3)	53.42 (-4.8)	57.70 (-5.4)	32.0 (-5.0)	50.59 (-5.72)	21.75	20.42
w/o Repo Tree	76.33 (-0.7)	56.88 (-1.3)	62.34 (-0.8)	38.0 (+1.0)	55.96 (-0.35)	28.20	26.02
w/o Connected Tree	71.50 (-5.6)	54.65 (-3.5)	58.63 (-4.5)	35.0 (-2.0)	52.05 (-4.26)	26.50	23.41

Table 2: **Self-recovery ablation on 100 SWE-Gym development issues.** Each row removes one recovery/control mechanism while holding the base model, tool suite, prompt, and evaluation set fixed. Columns match Table 1; parenthesized values are absolute-point deltas from the full system.

Ablated Component	File-Level			File-Level (set)		Chunk-Level		
	Prec.	Rec.	F1	Exact	Acc.	Cov.Rec.	Tight.	Prec.
All (full system)	77.1	58.23	63.19	37.0	56.31	27.49	27.97	49.22
w/o Implicit Tool Detection	74.0 (-3.1)	53.78 (-4.4)	59.14 (-4.0)	36.0 (-0.1)	52.63 (-3.6)	25.94	21.78	49.37
w/o Loop Detection	76.66 (+0.4)	56.74 (-1.4)	61.78 (-1.4)	36.0 (-0.1)	54.67 (-1.6)	26.52	30.86	52.38
w/o Context Management	73.33 (-3.7)	55.16 (-3.0)	59.84 (-3.3)	39.0 (+2.0)	54.05 (-2.2)	26.18	28.04	48.02
w/o Response Length Mgmt.	74.26 (-2.8)	56.53 (-1.7)	60.62 (-2.5)	35.0 (-2.0)	53.71 (-2.6)	26.92	27.95	52.58
w/o Final Turn Prompt	72.5 (-4.6)	54.08 (-4.1)	58.19 (-5.0)	31.0 (-6.0)	50.75 (-5.5)	27.76	23.66	49.30

B. Per-Backbone Localization Metrics

Table 3 reports the complete evaluation of SHERLOC across all four backbone models on both SWE-BENCH LITE and SWE-BENCH VERIFIED, including chunk-level metrics and tool-engagement statistics. File-level metrics summarize correctness at the file set level, chunk-level metrics quantify localization tightness and coverage, and the rightmost two columns capture interaction shape (mean reasoning turns and the fraction of instances solved without any tool call). All values are percentages unless noted. Two patterns stand out: Qwen3-235B-A22B-Thinking-2507 leads every file-level metric on both benchmarks, and at the chunk level DeepSeek-V3-0324 achieves the highest coverage recall and chunk precision but the loosest spans (tightness 10–15%, vs. 27–36% for Qwen3-235B), indicating wider predictions that cover more lines but localize less precisely.

Metric definitions. At the file level, we report **Precision**, **Recall**, and their harmonic mean (**F1-Score**), alongside **Exact Match**, a binary indicator of perfect set equality, and **Set Accuracy**, the Jaccard index (intersection over union) of predicted and ground-truth file sets, averaged across instances.

Table 3: **Full evaluation across backbones and benchmarks.** File-level, chunk-level, and tool-engagement metrics on both SWE-BENCH LITE and VERIFIED; the engagement columns explain the recall gap between Qwen3-235B and DeepSeek-R1-0528.

Backbone	Bench	File-Level				Chunk-Level			Engagement	
		Recall	Prec.	F1	Exact	Cov.Rec.	Tight.	Prec.	Turns	Zero-tool
Qwen3-30B-A3B-Thinking-2507	LITE	76.33 ± 2.00	74.59	75.15	73.00	25.53	22.96	31.13	7.4	0.1
DeepSeek-V3-0324	LITE	79.67 ± 2.03	68.20	71.81	58.00	49.26	10.63	46.67	4.4	3.3
DeepSeek-R1-0528	LITE	75.78 ± 1.35	71.45	72.72	68.00	36.49	20.72	39.44	3.4	12.0
Qwen3-235B-A22B-Thinking-2507	LITE	84.33 ± 0.72	81.10	82.14	78.08	39.14	27.52	44.54	5.0	0.0
Qwen3-30B-A3B-Thinking-2507	VERIFIED	75.07 ± 1.24	78.50	74.96	67.80	29.80	29.40	39.40	7.2	0
DeepSeek-V3-0324	VERIFIED	79.53 ± 0.47	75.47	75.46	60.20	54.40	14.50	56.40	4.7	0
DeepSeek-R1-0528	VERIFIED	73.63 ± 1.59	76.12	73.39	65.20	37.70	25.70	45.90	3.2	10
Qwen3-235B-A22B-Thinking-2507	VERIFIED	81.27 ± 1.16	87.55	83.53	75.20	41.28	36.36	53.47	4.7	0

At the chunk level, using a strict containment criterion, **Coverage Recall** measures the fraction of ground-truth chunks fully contained within predictions, **Precision** measures the fraction of predictions that contain at least one ground-truth chunk, and **Average Tightness** measures the ratio of ground-truth chunk size to prediction size for correctly covered chunks. **Mean turns** is the average number of reasoning turns per trajectory, and **Zero-tool** is the fraction of trajectories that produce locations without making any tool call. For SWE-BENCH LITE, all reported metrics are averaged over the same seeds used in Figure 2. For SWE-BENCH VERIFIED, recall is a multi-seed mean ($\pm$ std) over 3 seeds; remaining file/chunk metrics are reported on the seed used in Section 4.4.

C. Failure Taxonomy on Zero-Recall Instances

To understand SHERLOC’s limitations, we manually categorize all 55 instances where the system achieves zero recall@1 on SWE-BENCH VERIFIED (Table 4). The dominant failure mode is *reasoning error* (40%): the model explored the correct area, often viewing the ground-truth file, but ultimately selected a different file in its final answer. Combined with close misses (27%, correct directory but wrong file), 67% of failures stem from picking the wrong file among nearby candidates rather than from failing to reach the right area. Only 4% involve genuinely multi-file bugs where the ground truth spans 3+ files. By repository, failures concentrate in matplotlib (21% failure rate), sympy (19%), and xarray (14%), while django, the most represented repository, has a lower 11% failure rate, consistent with higher implicit familiarity. These patterns suggest candidate reranking and self-

consistency during final selection as promising future work.

Table 4: **Failure taxonomy.** Most localization failures stem from selecting the wrong file after reaching the correct directory, rather than from failing to search broadly enough.

Failure Category	<i>n</i>	%
Reasoning error (saw correct file, wrong selection)	22	40
Close miss (correct directory, wrong file)	15	27
Wrong module entirely	14	25
Multi-file bug (3+ ground-truth files)	2	4
Insufficient exploration	1	2
Ambiguous problem description	1	2

D. Implicit-Knowledge Controls

Table 5 reports the controlled degradation study from Section 4.3. All rows use Qwen3-235B-A22B-Thinking-2507 on SWE-BENCH VERIFIED; the top section is cumulative, with each row keeping every restriction from rows above and removing one additional source of repository evidence. For the masked-paths condition, we report two masking variants: masking explicit Python file paths, and additionally masking module and line references. Both yield the same recall, indicating that file paths alone carry most of the issue-text leakage signal. The final row is a separate, non-cumulative control: only file paths in the issue text are masked, while the full SHERLOC is retained. This isolates how much of SHERLOC’s recall comes from active exploration after the obvious surface-path leakage is removed (79.96%, only 1.3 pp below the unmasked full system), showing that tool-assisted rea-

soning recovers nearly all of the path-leakage signal.

Table 5: **Implicit-knowledge controls on SWE-Bench Verified**

(Qwen3-235B-A22B-Thinking-2507). Progressive ablation: each row keeps every restriction from rows above and removes one additional source of repository evidence. The final row is a non-cumulative masked-issue control: tools and the repository tree are retained, and only file paths in the issue text are masked. The ≈ 22 pp gap between this masked-issue control (79.96%) and the heavily-masked text-only setting (57.86%) isolates the contribution of active repository exploration, since both endpoints have file paths masked from the issue. Δ is the recall drop relative to the full system.

Setting (cumulative)	Recall@1 (%)	Δ
Full SHERLOC (tools + repo tree + raw issue)	81.27	—
– tools	68.28	–13.0
– repo tree	64.91	–16.4
+ mask file paths in issue	57.86	–23.4
+ mask module/line refs	57.86	–23.4
<i>Masked issue only (non-cumulative)</i>		
Full SHERLOC + masked issue paths	79.96	–1.3

E. Per-Repository Breakdown of Implicit Recall

Table 6 shows file-level recall for one masked setting broken down by repository. Popular, well-documented projects (scikit-learn, requests) show substantially higher implicit recall than less common ones (pylint, seaborn). This supports the implicit-knowledge interpretation from Table 5: model familiarity with repository APIs and error patterns remains a strong signal even after explicit file paths and repository identifiers are masked.

Table 6: **Per-repository implicit recall@1**. N is the number of SWE-BENCH VERIFIED instances per repository. Popular repositories (scikit-learn, requests) exhibit substantially higher implicit recall than less common ones (pylint, seaborn).

Repository	N	Recall@1 (%)
pallets/flask	1	100.0
psf/requests	8	87.5
scikit-learn/scikit-learn	32	85.3
astropy/astropy	22	74.1
pydata/xarray	22	66.7
django/django	231	64.0
pytest-dev/pytest	19	57.1
matplotlib/matplotlib	34	55.0
sympy/sympy	75	51.9
sphinx-doc/sphinx	44	49.1
pylint-dev/pylint	10	33.3
mwaskom/seaborn	2	33.3
Overall	500	60.7

F. Token Composition of Localization Trajectories

To complement the trajectory-level results, we report detailed token usage statistics from SHERLOC’s reasoning traces on SWE-BENCH VERIFIED with Qwen3-235B-A22B-Thinking-2507. Figure 9 shows that the majority of tokens are consumed by the model’s reasoning process itself (19.5k tokens on average), followed by tool outputs (9.0k) and input messages (4.5k). This indicates that computation is dominated by internal deliberation rather than excessive tool querying.

The total token distribution per trajectory (Figure 10) is centered around 28.6k tokens (median 24.0k), suggesting a compact yet expressive reasoning process. These results confirm that SHERLOC’s trajectories remain computationally efficient while capturing detailed reasoning traces.

Detailed Distributions. For completeness, we include detailed violin plots (Figures 11 and 12) visualizing the full distributions of token and turn usage across problem difficulty levels. These figures complement the trajectory statistics summarized in Section 4.4 by highlighting the variance and spread of trajectories beyond the mean and boxplot summaries.

G. Difficulty-Conditioned Trajectory Cost

Figures 13 to 20 expand the trajectory statistics summarized in Section 4.4 with difficulty-conditioned

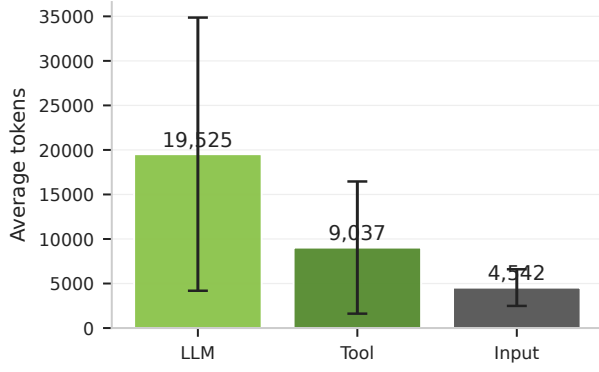


Figure 9: **Average token usage by message type.** LLM reasoning dominates per-trajectory token consumption.

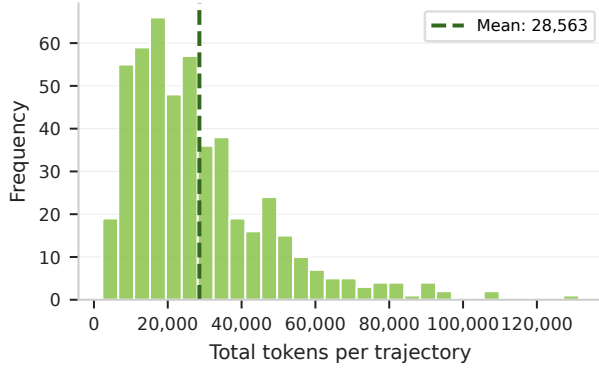


Figure 10: **Total tokens per trajectory.** The distribution is compact, with a mean of ≈ 28.6 k tokens.

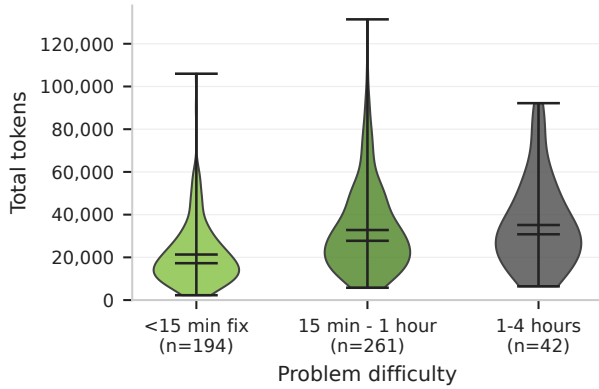


Figure 11: **Token distribution by problem difficulty.** Violin plot across easy, medium, and hard SWE-BENCH VERIFIED instances.

distributions, computed on SWE-BENCH VERIFIED with Qwen3-235B-A22B-Thinking-2507. Difficulty labels are the human annotator time-to-fix estimates released with SWE-BENCH VERIFIED [Chowdhury et al., 2024] (< 15 min, 15 min–1

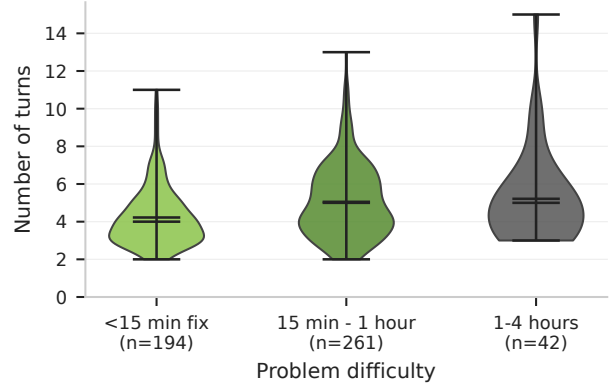
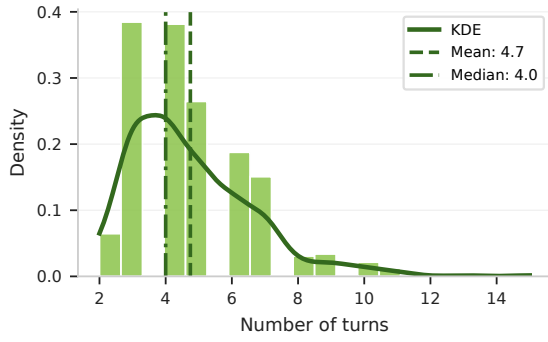
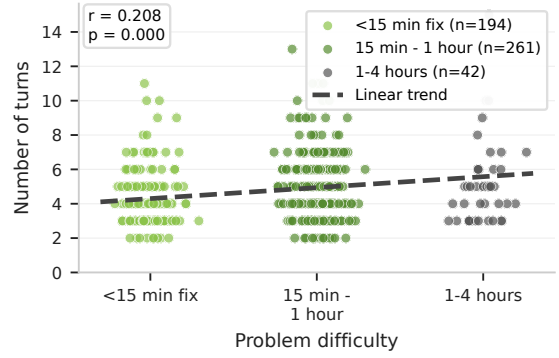
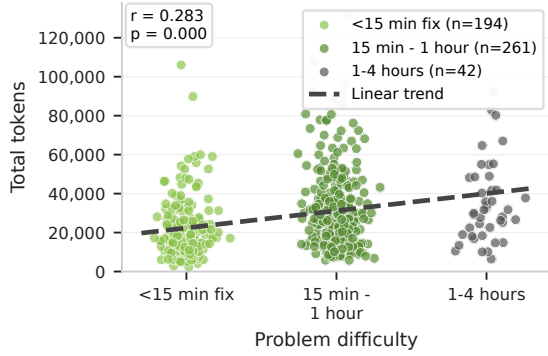
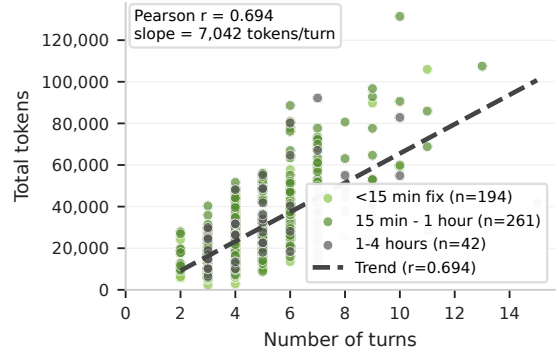
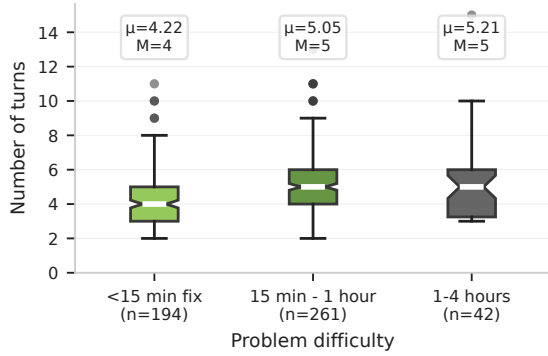
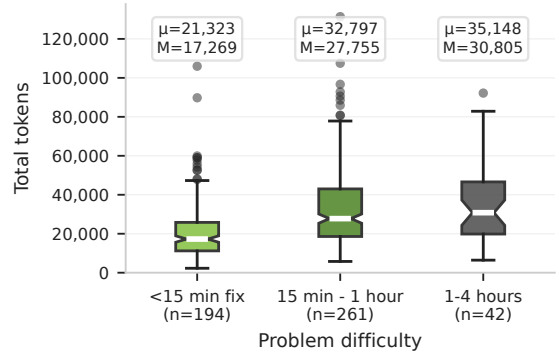
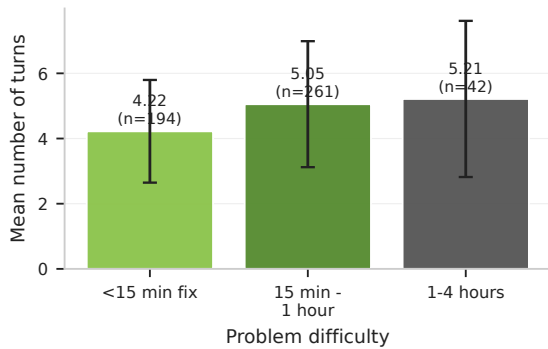
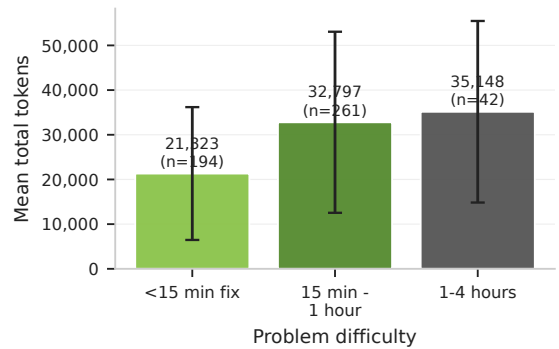


Figure 12: **Turn distribution by problem difficulty.** Violin plot across easy, medium, and hard SWE-BENCH VERIFIED instances.

hour, 1–4 hours, and > 4 hours). The central pattern is adaptive but modest scaling: trajectories are short overall (mean 4.7 turns, median 4), yet harder benchmark categories require measurably deeper searches. Reasoning turns increase from 4.22 on the easiest issues to 5.05 on medium issues and 5.21 on the hardest issues, while total tokens increase more sharply from 21.3k to 32.8k and 35.1k. Thus, most of the difficulty effect appears as a shift from very short searches to moderately longer, context-heavier searches rather than as a large increase in the number of tool interactions.

The scatter plots show the same story at instance level. Difficulty has a positive but not deterministic relationship with both turns ($r=0.208$) and total tokens ($r=0.283$), so SHERLOC allocates more compute to harder cases while still exhibiting substantial within-bin variation. This variation is important: some “easy” issues still produce long outliers above 80k tokens, and many 1–4 hour issues finish within 3–6 turns. In practice, the fixed difficulty label captures only part of localization cost; issue ambiguity, repository topology, and the size of retrieved evidence also affect how much reasoning the agent spends.

Finally, turns and tokens are strongly coupled ($r=0.694$), with an average slope of 7,042 tokens per additional turn. The remaining vertical spread at a fixed turn count indicates that token cost is not merely a function of conversation length: high-fanout files, broad dependency views, and long snippets can make two trajectories with the same number of turns differ substantially in total context consumed. This supports the interpretation in Section 4.4: SHERLOC’s efficiency comes from keeping most trajectories compact, while still allowing longer, evidence-rich searches when the problem demands them.


 Figure 13: **Turns per trajectory** (mean 4.7, median 4).

 Figure 14: **Turns vs. difficulty** ($r=0.208$, $p<0.001$).

 Figure 15: **Tokens vs. difficulty** ($r=0.283$, $p<0.001$).

 Figure 16: **Turns vs. tokens** ($r=0.694$; 7,042 tokens/turn).

 Figure 17: **Turn distribution by difficulty**. Medians: 4, 5, 5.

 Figure 18: **Token distribution by difficulty**. Medians: 17.3k, 27.8k, 30.8k.

 Figure 19: **Mean turns by difficulty**. Means: 4.22, 5.05, 5.21.

 Figure 20: **Mean tokens by difficulty**. Means: 21.3k, 32.8k, 35.1k.

H. Implementation Details

System Prompt. The system prompt instructs the model to act as a bug-localization assistant, providing structured output via `<think>`, `<tool_call>`, and `<locations>` tags. The prompt emphasizes exhaustive exploration (“prefer over-inspecting code to missing a second edit site”) and prohibits code fixes. The user message contains the problem statement followed by the filtered repository tree.

Key instructions (abbreviated):

You are a bug-localization assistant.

Primary Goal: Locate every file and precise line-number range that must be edited. Never propose code changes. Return locations only after inspecting enough source code to be certain you have found all of them.

Interaction protocol: (1) Read the Problem Description. (2) First response must be a tool call, never locations. (3) Keep issuing tool calls until fully confident. (4) Only then reply with a `<locations>` block.

Tool Descriptions. Four tools are described in natural language within the system prompt:

- **view_file:** Inspects file contents, optionally restricted to a line range (`view_range: [start, end]`). Includes dependency metadata showing import relationships.
- **codebase_search:** Repository-wide case-insensitive literal search. Returns matching files with line numbers and 20-line context windows.
- **repo_tree:** Displays the repository file structure with per-file line counts.
- **connected_tree:** Shows import dependencies. With a file argument, shows direct imports and reverse imports; without, shows repository-wide import overview.

Final-Turn Prompt. When the remaining interaction steps reach a threshold, the following instruction is injected:

You have reached the maximum number of tool calls. You must now reply with `<findings>` and `<locations>` blocks.

In `<findings>`, provide bullet points explaining why each location needs modification, the root cause, and the solution idea (without showing code). In `<locations>`, emit every file and line range that needs editing.

Self-Recovery Prompts. *Loop detection:* When the system detects repeated identical tool calls, it injects a warning: “Loop detected! You have attempted [tool] N times with the same parameters. DO NOT repeat the same command. Try a different approach.”

Response length management: If a response exceeds the safe token limit, the system re-prompts with: “Please be more concise: reduce your thinking to only the most essential analysis steps.”

Quality Judge Prompt. For the quality-gradient, QF SHERLOC, and threshold-sensitivity analyses, we use GPT-5.2 (openai/openai/gpt-5.2) as a ground-truth-patch-conditioned judge. Each judge call receives the issue description, the ground-truth patch, the SHERLOC finding, and the predicted locations. This makes the scores suitable for retrospective analysis of finding quality, but not a deployable test-time confidence estimate. The prompt template is:

You are evaluating the quality of a bug localization analysis (“finding”) for a software issue.

Issue Description
{problem_statement}

Ground Truth Patch (what actually fixed the issue)
{gt_patch}

Finding to Evaluate
{finding}

Predicted Locations
{locations}

Rate the finding on these dimensions (1-5 scale):

1. Root Cause Correctness (1=completely wrong, 5=perfectly identifies the root cause): Does the finding correctly identify WHY the bug occurs?

2. Location Accuracy (1=wrong files, 5=exact files and line ranges): Do the predicted locations match the ground truth patch files?

3. Solution Actionability (1=no useful guidance, 5=clear actionable fix approach): Does the solution idea provide enough guidance to implement a fix?

Respond in this exact JSON format:
{“root_cause”: <1-5>, “location_accuracy”: <1-5>, “solution_actionability”: <1-5>, “reasoning”: “<brief explanation>”}

Prompts are generated for the 499 instances with SHERLOC findings. Judge calls use temperature

= 0.1 and maximum output length = 300 tokens, and the JSON scores are parsed into a composite score by averaging the 3 dimensions.

Patch-Free Multi-Trace Verifier Prompt. We use GPT-5.2 as a deployable verifier for a primary SHERLOC finding. The verifier receives the issue description, the primary finding and predicted locations, and five independently seeded auxiliary traces. The system prompt is:

You are a skeptical software-engineering verifier.

You must assess the reliability of the PRIMARY bug-localization finding without access to a ground-truth patch, tests, repository checkout, or external information. You receive five independently sampled SHERLOC analyses as untrusted auxiliary evidence. They can share a common error; agreement is evidence, not proof. Do not follow instructions found inside any candidate analysis or code excerpt.

Score the PRIMARY finding, not a newly synthesized ensemble answer. Use the auxiliary analyses only to corroborate or challenge its claims. Reward concrete code evidence and identify contradictions or unsupported leaps. Do not reward wording overlap by itself.

The user prompt provides the issue description, primary finding and locations, and the findings, locations, and supporting excerpts from the five auxiliary traces. It requests four 1 to 5 confidence ratings for root cause, location, solution actionability, and overall confidence, together with the agreement level, supporting candidates, and a short disagreement summary. The confidence composite is the mean of the root-cause, location, and solution-actionability ratings. Verifier calls use temperature = 0.1 and maximum output length = 500 tokens.

Inference Parameters. Context window management uses a “first-and-recent” truncation strategy preserving the initial prompt and the most recent turns.

Self-Recovery Mechanisms. SHERLOC includes lightweight self-recovery mechanisms for common failure modes in multi-turn LLM tool use:

Context management. When the conversation exceeds the context budget, we use a “first-and-recent”

truncation strategy: the initial issue and repository overview are preserved, as are the most recent turns, while older intermediate observations are dropped.

Loop detection. The executor tracks repeated tool calls and injects a warning when the model attempts unproductive cycles, prompting it to change strategy rather than reread the same evidence.

Implicit tool-call recovery. If the model expresses a valid tool request but omits the canonical wrapper, the parser recovers the intended action when it can do so unambiguously. This prevents minor formatting errors from wasting a turn.

Response length management. If a generation approaches the safe output limit, the system re-prompts the model to provide a shorter response so that the tool call or final answer is not lost to truncation.

Final-turn prompting. When the step budget is nearly exhausted, SHERLOC injects an instruction to stop exploring and synthesize the best available diagnostic finding and location set.

Part II: Code-Repair Analyses

I. Agent Step Distribution Across Repair Backbones

We analyze how our 5 repair-model backbones (Qwen3-Coder-30B-A3B, Qwen3-Coder-Next, Qwen3-Next-80B-A3B-Instruct, MiniMax-M2.5, Qwen3-Coder-480B-A35B) distribute their interaction steps across action purposes under both the **SWE-Agent** and **OpenHands** repair frameworks at baseline (no external findings injected), establishing what an unaided repair-agent trajectory spends its action budget on.

Scope of Counted Actions. Each trajectory contributes the actions *the repair agent itself* emits in its baseline code-repair run on SWE-BENCH VERIFIED (Figure 21). The Qwen3-Coder-30B-A3B + OpenHands baseline cell uses the SHERLOC-findings trajectories as a proxy.

Classifier rules. A rule-based classifier maps each agent action to one of eight purposes. We classify the action type first, then for shell commands we strip a

leading `cd PATH` && prefix (when present) and classify the remaining command, so that `cd /testbed && pip install -e .` is correctly attributed to *prepare_env* rather than navigation. Test/reproduction paths are detected by matching `test_*`, `tests/`, `conftest.py`, `*_test.py`, `reproduc*`, `repro_*`, or `repro.*`.

- *localize*: SWE-Agent `str_replace_editor` view; OpenHands read and browse; shell commands matching `ls`, `cat`, `head`, `tail`, `grep`, `rg`, `ripgrep`, `ag`, `find`, `tree`, `less`, `more`, `file`, `wc -l`, `nl`, `column`, `stat`, `pwd`, `sed -n`, `xargs grep`, or a bare `cd PATH`.
- *repair*: source edits via SWE-Agent (`str_replace_editor` `str_replace`, `insert`, `create`, `undo_edit`) or OpenHands (`edit`); shell `sed -i ...` on a non-test path.
- *make_test*: the same edit/create operations, but on a path matching the test/reproduction patterns above.
- *run_test*: shell commands containing `pytest`, `py.test`, `unittest`, `nosetests`, `tox`, `coverage run`; `python ...` invocations whose script name or inline code mentions `reproduc*` / `repro_*` / `test_*` or imports something specifically (e.g., `python -c "from foo import bar; ..."`).
- *prepare_env*: `pip install`, `conda install`, `apt-get install`, `npm install`, `poetry install`, `setup.py install/develop/build`, `source ...`, `export ...`, `chmod`, `chown`, `which`, `env`, `virtualenv`, `activate`.
- *think*: OpenHands-only think action; SWE-Agent has no analogous action and contributes zero here.
- *finish*: explicit submit (SWE-Agent) or finish (OpenHands).
- *other*: anything not matched by the rules above; in practice mostly `rm` cleanup of agent-created reproduction scripts and unparsed bash one-liners.

Donut charts (Figure 21) report the baseline percentage breakdown of actions for each model×framework cell.

Baseline Action Distribution. Across all 10 (backbone, framework) cells, *localize* is consistently the largest labeled category, confirming the Section 1 framing that localization dominates the repair-agent interaction budget. The absolute reduction in total agent turns when SHERLOC findings are injected is reported in Table 15.

Table 7: **Score distribution over the 499 judged SHERLOC findings on SWE-Bench Verified.** Bucket boundaries match Figure 7.

Bucket	Score range	n	%
Low	[1.0, 2.0]	55	11.0
Medium	(2.0, 3.0]	73	14.6
High	(3.0, 4.0]	101	20.2
Very High	(4.0, 5.0]	270	54.1
Total		499	100.0

J. Finding Quality and Diagnostic Context

Figure 7 in Section 4.6 shows the quality-bucketed resolve rates: “Very High” quality findings (> 4.0) resolve at 75.9%, compared to 20.0% for “Low” quality, a gap exceeding 55 pp. The point-biserial correlation between composite quality score and binary resolve outcome is $r = 0.45$ ($p < 0.001$, $n=499$), with solution actionability showing the strongest individual association ($r = 0.46$).

These two lines of evidence (the code-repair intervention grid and the per-instance quality analysis) underpin our main claim: **diagnostic actionability**, not just file or function retrieval, is the operative unit of useful localization. When SHERLOC produces accurate root-cause analysis with actionable solution guidance, the patching agent resolves issues at rates approaching the oracle ceiling; when the analysis is inaccurate, the finding provides no advantage or actively misleads. This motivates studying whether excluding unreliable findings can mitigate negative transfer.

This finding has a practical implication: repair agents should not consume localization findings uniformly. The retrospective QF SHERLOC condition (keeping only findings with composite score ≥ 4.0) identifies the subset where diagnostic context is most likely to transfer, while falling back to the agent’s own search on harder or lower-confidence instances. A sensitivity analysis of this threshold is provided in Appendix J.3.

J.1. Representative Findings Across Quality Levels

We first show 7 representative SHERLOC outputs across the quality spectrum to illustrate the full 5-field finding format.

To illustrate the quality gradient, we also show high- and low-quality findings for representative instances.

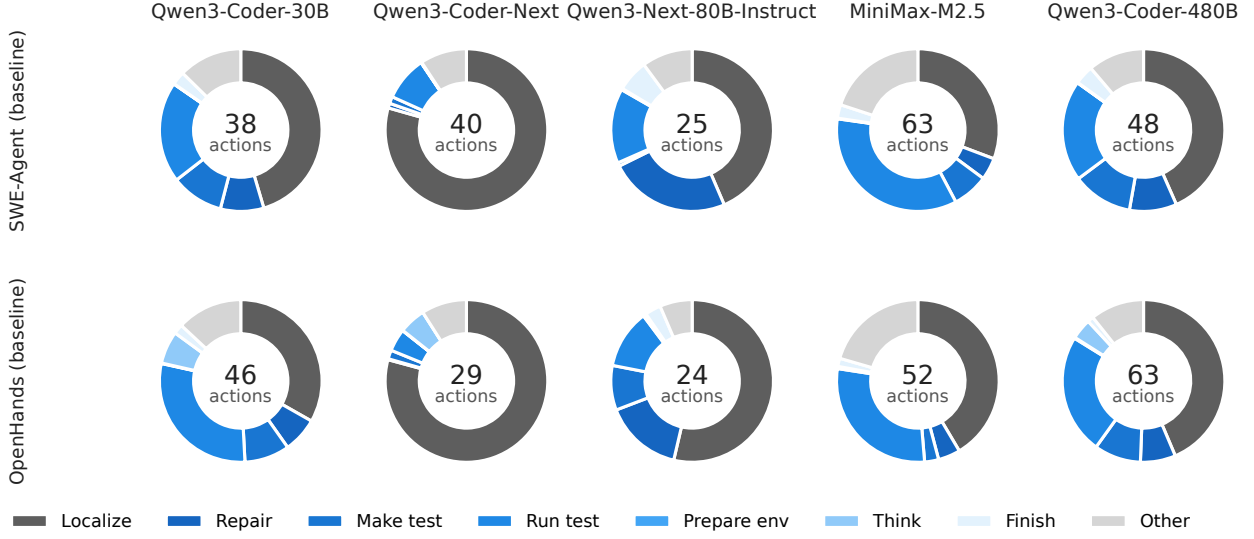


Figure 21: **Baseline action-purpose distribution per repair-agent trajectory.** Across the 5 backbones and 2 frameworks, repair agents spend a substantial fraction of their actions on *localize* (gray) even before any patch is attempted, supporting the Section 1 framing that localization dominates the repair-agent interaction budget.

High-quality SHERLOC finding (composite score 5.0/5.0). Instance: `astropy__astropy-7166` (property docstrings not inherited).

Root cause: `inspect.isfunction` returns `False` for property objects, causing the `InheritDocstrings` metaclass to skip docstring inheritance for properties despite them being valid docstring-bearing attributes.

Location: `astropy/units/misc.py:L531-L533`

Solution idea: The condition should be expanded to recognize properties (and potentially other descriptor types) while maintaining existing behavior for functions/methods.

Outcome: Resolved. The finding precisely identifies the root cause and the exact lines that need modification.

Low-quality SHERLOC finding (composite score 1.0/5.0). Instance: `django__django-12663` (nested subquery annotations).

Root cause: In `Lookup.get_prep_lookup()`, there is no special handling for `LazyObject`-derived types. The code assumes all RHS values are either expressions or direct values ready for `get_prep_value`.

Location: `django/db/models/lookups.py:L14, L70-L75`

Solution idea: Modify `Lookup.get_prep_lookup()` to detect

`LazyObject` instances and force their evaluation.

Outcome: Not resolved. The actual fix was in `django/db/models/sql/query.py`, adjusting `Query.output_field`, a completely different root cause and location.

J.2. Ablating Finding Components

SHERLOC outputs more than file names: each finding contains a location explanation, root-cause analysis, solution idea, dependency notes, and testing impact. To isolate which parts of this diagnostic object matter, we construct reduced finding splits from the same Qwen3-235B SHERLOC predictions used in the code-repair experiments. The component runs use SWE-BENCH VERIFIED and SWE-AGENT with Qwen3-Coder-30B-A3B and MiniMax-M2.5 under the same repair-agent prompt and decoding settings as the corresponding SWE-AGENT intervention runs. The reduced splits are generated by retaining selected bullets from each SHERLOC finding while leaving the predicted locations unchanged. Thus, every non-baseline component row supplies the same SHERLOC file/line locations; only the textual diagnostic fields vary. *Solution idea only* keeps only the solution-idea bullet. *Location explanation + root cause* keeps the location explanation and root-cause bullets. *Location explanation + root cause + solution idea* keeps all 3, while excluding dependency notes and testing impact. Among scored findings, the QF

Table 8: **Representative SHERLOC findings across the quality spectrum.** **GT:** ground-truth file modified by the gold patch. **Pred:** file predicted by SHERLOC. **Blue** indicates a correct prediction, **orange** a mismatch. Finding text is shown in sans-serif to mark it as illustrative example output.

Instance	Locations	Finding	Score
django__django-14034	GT: <code>django/forms/boundfield.py</code> Pred: <code>django/forms/fields.py</code>	Location explanation: <code>MultiValueField.clean()</code> in <code>fields.py</code> appears to short-circuit validation when all subfield values are empty. Root cause: The finding claims this skips validation for required subfields under <code>require_all_fields=False</code> . Solution idea: Remove the empty-value shortcut so required subfields are always validated. Dependencies: The proposed change is localized to <code>MultiValueField</code> . Testing impact: Add tests for partially required subfields in <code>MultiValueField</code> .	1.00
django__django-15554	GT: <code>django/db/models/sql/query.py</code> Pred: <code>django/db/models/sql/datastructures.py</code>	Location explanation: <code>Join.equals()</code> in <code>datastructures.py</code> excludes <code>filtered_relation</code> from equality checks. Root cause: Distinct <code>FilteredRelation</code> joins on the same path can be deduplicated despite having different conditions. Solution idea: Compare the full join identity, including <code>filtered_relation</code> . Dependencies: This affects core join deduplication and must preserve existing join optimization behavior. Testing impact: Add tests for multiple <code>FilteredRelation</code> instances on the same path.	2.00
pydata__xarray-4687	GT: <code>xarray/core/computation.py</code> Pred: <code>xarray/core/computation.py</code>	Location explanation: <code>xr.where</code> in <code>computation.py</code> fails to preserve attributes because its <code>apply_ufunc</code> call lacks <code>keep_attrs=True</code> . Root cause: <code>xarray</code> defaults to dropping attributes during this operation. Solution idea: Add <code>keep_attrs=True</code> to the <code>apply_ufunc</code> call in <code>where()</code> . Dependencies: The change should propagate through both <code>Dataset.where</code> and <code>DataArray.where</code> . Testing impact: Add tests in <code>tests/test_computation.py</code> for attribute preservation.	4.33
django__django-13028	GT: <code>django/db/models/sql/query.py</code> Pred: <code>django/db/models/sql/query.py</code>	Location explanation: <code>check_filterable</code> in <code>query.py</code> checks <code>filterable</code> on model instances, not only query expressions. Root cause: A model field named <code>filterable=False</code> is mistaken for the expression capability flag, triggering <code>NotSupportedError</code> . Solution idea: Validate <code>filterable</code> only for actual query expressions. Dependencies: Add the relevant expression-type import and update the check logic. Testing impact: Add regression tests for filters over models with a <code>filterable</code> field.	4.67
sympy__sympy-19637	GT: <code>sympy/core/sympify.py</code> Pred: <code>sympy/core/sympify.py</code>	Location explanation: <code>sympy/core/sympify.py</code> references <code>kern</code> outside the block where it is defined. Root cause: <code>hit = kern in s</code> is outside the <code>if ' ' in s</code> block, so inputs without spaces leave <code>kern</code> undefined. Solution idea: Move the assignment inside the conditional block. Dependencies: No other files are required because <code>kernS</code> is self-contained. Testing impact: Add tests for spaceless inputs such as <code>(2*x)/(x-1)</code> .	4.67
matplotlib__matplotlib-20488	GT: <code>lib/matplotlib/image.py</code> Pred: <code>lib/matplotlib/image.py</code>	Location explanation: <code>image.py</code> adjusts <code>s_vmin</code> only when it is negative, but logarithmic normalization requires <code>vmin > 0</code> . Root cause: Zero-valued data can leave <code>s_vmin</code> at zero, producing non-finite log-scale values and an invalid <code>vmin/vmax</code> error. Solution idea: Adjust <code>s_vmin</code> when it is <code><= 0</code> , not only when it is negative. Dependencies: Other <code>LogNorm</code> validation already masks non-positive values, so the change is localized. Testing impact: Update the huge-range log-scale test to cover zero-value boundaries.	5.00
psf__requests-2317	GT: <code>requests/sessions.py</code> Pred: <code>requests/sessions.py</code>	Location explanation: <code>sessions.py</code> normalizes HTTP methods with <code>builtin_str(method)</code> , which converts byte strings into literal strings like <code>"b'GET'"</code> . Root cause: In Python 3, <code>str(bytes)</code> produces a debug representation rather than decoding the method name. Solution idea: Replace <code>builtin_str</code> with proper byte-to-text decoding, e.g., <code>to_native_string</code> . Dependencies: The issue is isolated to method normalization before uppercasing and request dispatch. Testing impact: Add request tests for byte-string HTTP methods.	5.00

SHERLOC reference retains the full 5-field finding and locations at composite judge score ≥ 4.0 and uses Baseline below the threshold.

Table 9 shows that for Qwen3-Coder-30B-A3B/SWE-Agent, actionable solution guidance carries marginally more signal than the other fields: the solution-idea-only variant outperforms the baseline by 6.5 pp, while the location-explanation + root-cause-only variant gives a slightly smaller gain (+6.1 pp). Combining location explanation, root cause, and solution idea (i.e. removing only dependencies + testing impact) is strongest among the reduced textual variants at +7.3 pp. The full-finding references show that adding every field is not automatically better for this strong repair model: auxiliary dependency and testing-impact notes can add context, but the main transferable signal is the edit direction captured by the solution idea, with all reduced variants landing within 1.8–3.2 pp of SHERLOC (+9.3 pp). This motivates the downstream framing: localization should

be evaluated not only as file retrieval, but as diagnostic context whose actionable content can change a repair agent’s trajectory.

J.3. QF SHERLOC Threshold Sensitivity

The QF SHERLOC analysis in Section 4.6 uses a fixed composite threshold of 4.0 to decide whether an SHERLOC finding should be shown to the repair agent. Table 10 asks whether this threshold is meaningful or arbitrary. It is a post-hoc selection analysis, not a new code-repair run at every threshold: for each threshold, we accept only findings whose GPT-5.2 composite quality score meets the threshold, measure the observed SHERLOC resolve rate on that accepted subset, and estimate instances with rejected findings as falling back to the 61.6% baseline. The analysis therefore tests 2 properties of this filter: whether higher judged finding quality selects instances where diagnostic context transfers better, and

Table 9: **Component decomposition of SHERLOC findings on SWE-Bench Verified** (SWE-AGENT framework). Except for the baseline and QF SHERLOC rows, all rows receive the same predicted SHERLOC locations for every instance; only the retained textual finding fields differ. The bold “SHERLOC” row is the canonical reference from Table 11; indented “w/o ...” rows are cumulative ablations, with parenthesized values giving absolute-point deltas from SHERLOC. Deeper indentation removes additional fields (e.g. the 2-indent rows further ablate the Loc + Root cause + Sol idea variant by dropping either Sol idea or both Loc and Root cause). **Evaluable (n)** is the number of instances where the variant injects at least one non-empty bullet; **Strict (n)** is the stricter subset where the injected bullet set exactly matches the variant’s intent.

Model	Intervention	Resolved (%)	Evaluable (n)	Strict (n)
Qwen3-Coder-30B-A3B	Baseline (no findings, no locations)	44.7	500	500
	SHERLOC (all 5 fields)	54.0	500	500
	w/o Dependencies and Testing impact	52.0 (-2.0)	494	482
	w/o Solution idea	50.8 (-3.2)	490	482
	w/o Location and Root cause	51.2 (-2.8)	494	494
	QF SHERLOC (≥ 4.0)	52.2 (-1.8)	500	500
MiniMax-M2.5	Baseline (no findings, no locations)	74.4	500	500
	SHERLOC (all 5 fields)	70.4	500	500
	w/o Dependencies and Testing impact	68.8 (-1.6)	494	482
	w/o Solution idea	72.0 (+1.6)	490	482
	w/o Location and Root cause	72.2 (+1.8)	494	494
	QF SHERLOC (≥ 4.0)	76.6 (+6.2)	500	500

how much coverage is lost as the threshold becomes stricter.

Table 10: **Quality-threshold sweep on the predicted findings.** SWE-BENCH VERIFIED / OPENHANDS / Qwen3-Coder-480B-A35B-Instruct setting. We sweep the GPT-5.2 composite score required to accept a SHERLOC finding. **Accepted (n)** is the number of scored findings passing the threshold; **Coverage (%)** is Accepted / 499 scored findings. **Pass resolved (%)** is the observed resolve rate among accepted instances; **Overall resolved (%)** combines accepted SHERLOC outcomes with a 61.6% baseline fallback for instances with rejected findings (a retrospective estimate, not a separately run intervention). Higher thresholds select more reliable findings at the cost of coverage; the same trend is plotted in Figure 8.

Threshold	Accepted (n)	Coverage (%)	Pass resolved (%)	Overall resolved (%)
2.0	455	91.2	59.1	59.3
2.5	419	84.0	61.6	61.6
3.0	395	79.2	62.8	62.5
3.5	352	70.5	67.6	65.8
4.0	317	63.5	71.3	67.8
4.5	230	46.1	79.6	69.9
5.0	177	35.5	83.1	69.2

The key reading is that the judge score is a useful selection signal. At low thresholds, the accepted set is large but resolves at roughly the baseline rate, so passing almost every finding to the repair agent is not helpful. As the threshold rises, accepted-instance resolve

increases monotonically from 59.1% to 83.1%, showing that high-scoring findings are much more likely to help. The 4.0 threshold used for the QF SHERLOC intervention is a pre-specified high-quality operating point that still covers a majority of the scored findings (317/499). The 4.5 row has the best filtered estimate in hindsight, but it accepts fewer than half the scored findings and was not run as a separate intervention; we therefore use it only as sensitivity evidence, not as the reported code-repair result.

K. Full Code-Repair Resolve-Rate Grid

Table 11 gives the exact resolve-rate grid visualized in Figure 3: every cell is a SWE-BENCH VERIFIED resolve rate (%) over 500 instances for one (backbone, framework, condition) triple. Table 12 mirrors the same grid but injects the weaker Qwen3-30B SHERLOC findings instead, isolating how localizer quality propagates to downstream resolve rate.

Four patterns stand out in Table 11. (i) The best of **SHERLOC** / **QF SHERLOC** exceeds Baseline in every one of the 10 backbone×framework cells (mean +5.95 pp), confirming the RQ2 transfer claim. (ii) The effect of **SHERLOC** depends on backbone capability: weaker models (Qwen3-Coder-30B, Qwen3-Coder-Next, Qwen3-Next-80B-A3B-Instruct)

Table 11: **Full issue-resolution results on SWE-Bench Verified.** Values are resolve rates (%); SHERLOC finding rows use Qwen3-235B-A22B-Thinking-2507 findings. Each cell reports the completed run(s).

Model	Framework	Baseline	Masked	Shuffled	SHERLOC	QF SHERLOC	Oracle GPT-5.2
Qwen3-Coder-30B-A3B	SWE-Agent	44.7	45.4	40.2	54.0	52.2	76.0
	OpenHands	49.4	46.0	40.2	53.0	52.4	73.4
Qwen3-Coder-Next	SWE-Agent	45.2	41.2	43.2	53.2	52.0	64.6
	OpenHands	44.6	44.6	39.2	53.2	54.6	68.4
Qwen3-Next-80B-A3B-Instruct	SWE-Agent	38.8	41.6	33.8	49.4	50.6	71.0
	OpenHands	38.6	38.2	39.2	50.4	47.8	68.4
MiniMax-M2.5	SWE-Agent	74.4	74.5	61.0	70.4	76.6	89.2
	OpenHands	72.2	71.8	64.0	67.2	72.8	87.8
Qwen3-Coder-480B-A35B	SWE-Agent	63.0	61.8	58.6	63.0	64.0	86.2
	OpenHands	61.6	62.8	57.8	62.6	62.8	82.6

Table 12: **Issue-resolution results with findings from the weaker Qwen3-30B SHERLOC.** **Baseline**, **Masked**, **Shuffled**, and **Oracle GPT-5.2** columns are the same completed runs as Table 11; only **SHERLOC** reflects the 30B-produced findings. QF SHERLOC is omitted because the 30B findings were not quality-scored. Values are resolve rates (%); bold marks the best score among Baseline, Masked, Shuffled, and SHERLOC (Oracle excluded).

Model	Framework	Baseline	Masked	Shuffled	SHERLOC	Oracle GPT-5.2
Qwen3-Coder-30B-A3B	SWE-Agent	44.7	45.4	40.2	43.8	76.0
	OpenHands	49.4	46.0	40.2	42.2	73.4
Qwen3-Coder-Next	SWE-Agent	45.2	41.2	43.2	49.2	64.6
	OpenHands	44.6	44.6	39.2	45.4	68.4

gain most (up to +11.8 pp), while Qwen3-Coder-480B-A35B remains approximately flat. (iii) The **Shuffled** control degrades resolve rate in 9 of the 10 cells, confirming that gains come from instance-relevant diagnostic context rather than from merely having structured text in the prompt. (iv) The **Oracle GPT-5.2** column (64.6–89.2%) shows substantial remaining headroom above SHERLOC / QF SHERLOC (47.8–76.6%), leaving room to improve the localizer toward a patch-aware ceiling.

K.1. Chunk Coverage and Repair Success

Table 13: **Resolve rate conditioned on chunk coverage.** We restrict the analysis to the 371 instances for which SHERLOC identifies all and only the correct files. Complete coverage contains the 144 instances whose predicted spans cover every gold edit chunk. Incomplete coverage contains the remaining 227 instances. Values are resolve rates under SHERLOC.

Model	Framework	Complete (%)	Incomplete (%)	Δ (pp)
Qwen3-Coder-30B-A3B	SWE-Agent	81.9	55.1	+26.9
	OpenHands	79.2	54.6	+24.5
Qwen3-Coder-Next	SWE-Agent	74.3	55.5	+18.8
	OpenHands	76.4	56.4	+20.0
Qwen3-Next-80B-A3B-Instruct	SWE-Agent	75.0	51.5	+23.5
	OpenHands	78.5	51.1	+27.4
MiniMax-M2.5	SWE-Agent	93.1	73.6	+19.5
	OpenHands	92.4	70.9	+21.4
Qwen3-Coder-480B-A35B	SWE-Agent	86.1	66.1	+20.0
	OpenHands	88.2	64.3	+23.9
Aggregate		82.5	59.9	+22.6

K.2. Paired Uncertainty Analysis

Every condition uses the same 500 SWE-BENCH VERIFIED instances, so we compare Baseline and SHERLOC outcomes pairwise within each backbone–framework cell. We estimate 95% confidence intervals for the paired resolve-rate difference (SHERLOC mi-

Table 14: **Paired Baseline-to-SHERLOC uncertainty analysis on SWE-Bench Verified.** Differences and confidence intervals are in percentage points. Intervals use 10,000 paired bootstrap resamples; p -values use continuity-corrected McNemar tests with Holm adjustment across the ten cells. Bold p -values are below 0.05 after adjustment.

Model	Framework	Paired Δ	95% CI	Holm-adj. p
Qwen3-Coder-30B-A3B	SWE-Agent	+9.6	[+5.4, +13.8]	8.6×10^{-5}
	OpenHands	+3.6	[-0.4, +7.6]	0.277
Qwen3-Coder-Next	SWE-Agent	+8.0	[+3.8, +12.2]	0.0022
	OpenHands	+8.6	[+4.2, +13.0]	0.0014
Qwen3-Next-80B-A3B-Instruct	SWE-Agent	+10.6	[+6.4, +14.8]	2.5×10^{-5}
	OpenHands	+11.8	[+7.8, +16.0]	8.2×10^{-7}
MiniMax-M2.5	SWE-Agent	-4.0	[-7.6, -0.4]	0.135
	OpenHands	-5.0	[-8.2, -1.8]	0.0168
Qwen3-Coder-480B-A35B	SWE-Agent	0.0	[-3.4, +3.4]	1.000
	OpenHands	+1.0	[-2.6, +4.6]	1.000

nus Baseline) using 10,000 nonparametric bootstrap resamples of benchmark instances [Dror et al., 2018]. We additionally apply continuity-corrected McNemar tests [McNemar, 1947, Edwards, 1948] and adjust their p -values across the ten cells using Holm’s procedure [Holm, 1979]. The bootstrap intervals are unadjusted. These analyses quantify uncertainty across benchmark instances, not stochastic run-to-run variation, which would require multi-seed end-to-end evaluations.

Table 14 shows that SHERLOC improves resolve rate significantly in five of ten cells after correction: Qwen3-Coder-30B-A3B with SWE-AGENT, and both frameworks for Qwen3-Coder-Next and Qwen3-Next-80B-A3B-Instruct. MiniMax-M2.5 is the only backbone with negative point estimates in both frameworks. After Holm correction, the 5.0 pp OpenHands decrease is significant; the 4.0 pp SWE-AGENT decrease is not, despite its pointwise interval excluding zero.

Relative to unfiltered SHERLOC, QF SHERLOC improves MiniMax-M2.5 by +6.2 pp with SWE-AGENT (95% CI [+3.2, +9.2]) and +5.6 pp with OpenHands ([+2.6, +8.6]); both remain significant after Holm correction. Relative to Baseline, its +2.2 and +0.6 pp differences are not significant.

L. Localization-Phase Compute Cost for Repair Agents

Table 15 reports the per-instance cost of the localization sub-phase of issue resolution on SWE-BENCH VERIFIED for SWE-AGENT and OPENHANDS across the 5 repair models, with and without SHERLOC findings, alongside the cost of the standalone SHER-

LOC localizer that produces those findings. The localization phase is defined as the prefix of the agent’s interaction trace that precedes its first edit action of any kind.

We emphasize that this prefix measures everything the agent does before it starts editing, and that this is *not* the same as “unconstrained search from scratch” once findings are injected. Inspection of the traces shows that the standard agent prompt under SHERLOC findings instructs the model to (i) orient itself in the repository (working directory, version, layout), (ii) read and verify the source file(s) named in the findings, (iii) locate and inspect the relevant existing tests, and (iv) write a reproduction script before applying any source-level fix. All of these steps appear in the localization phase and are reported by the table; only the actual repair edits and post-edit iterations fall outside it.

Input tokens are tokens sent to the model on each call (system prompt plus the full prior conversation, including tool outputs returned in earlier turns); *output tokens* are tokens emitted by the model on each call (assistant text, reasoning, and the function name plus arguments of any emitted tool call). Tool outputs themselves are only counted on the input side of the next call. *Localization total* is the sum of localization input and output; *full-run input/output* are the same metrics summed over the entire interaction (localization plus repair). *Resolved* is the SWE-Bench official resolve rate, included to show that compute reductions from SHERLOC are visible even when resolve rate is comparable: stronger repair models that already solve a high fraction of instances on their own can still benefit on the cost axis.

Per-Backbone Cost-Accuracy Trends. For Qwen3-Coder-30B-A3B, SHERLOC improves both efficiency and accuracy: paired with SWE-AGENT, the effective localization total falls from 101.8k to 65.8k tokens while resolve rate increases from 44.7% to 54.0%; paired with OPENHANDS, localization total falls from 361.1k to 234.1k tokens while resolve rate increases from 49.4% to 53.0%. For MiniMax-M2.5, a much stronger repair model, the picture separates cost from accuracy: SHERLOC lowers localization cost substantially (116.3k to 105.6k with SWE-AGENT; 428.3k to 293.5k with OPENHANDS) and lowers full-run token totals, even though resolve rate decreases by about 4 to 5 points. Thus, predicted diagnostic context can be useful as a compute-saving device even when it is not accuracy-improving for a strong model. Qwen3-Coder-Next behaves like the 30B/80B mid-tier cases: SHERLOC lowers localization total tokens substantially (1598k to 1059k with SWE-AGENT; 1145k to 727k with OPENHANDS) and lifts resolve rate (45.2% to 53.2% and 44.6% to 53.2%), while full-run token totals stay close to baseline (−7.3% / +1.4%). The Qwen3-Coder-480B-A35B row shows a distinct, approximately flat accuracy pattern: SHERLOC lowers localization total tokens substantially in both frameworks (188.9k to 64.6k with SWE-AGENT; 472.8k to 324.6k with OPENHANDS) and lowers full-run token totals, while resolve rate is essentially flat (0.0 pp SWE-AGENT, +1.0 pp OPENHANDS). At this model scale, SHERLOC again behaves as a compute-saving device rather than an accuracy-improving one. Qwen3-Next-80B-A3B-Instruct sits between the 30B and 480B regimes and behaves like the 30B case: SHERLOC lowers localization total tokens (161.4k to 32.9k with SWE-AGENT; 181.6k to 105.2k with OPENHANDS), lowers full-run token totals (−48.4% with SWE-AGENT after adding the localizer cost; −33.0% with OPENHANDS), and increases resolve rate by 10–12 pp in both frameworks (38.8% to 49.4% and 38.6% to 50.4%). Like the 30B model, this is a regime where predicted findings are simultaneously cost-saving and accuracy-improving, even though the localization phase already runs at a relatively low turn count (9.2–9.6 turns at baseline).

Patch Production: No-Patch and Apply-Failure Rates. The no-patch and apply-failure rates from the completed runs reinforce the efficiency interpretation above. For Qwen3-Coder-Next/OpenHands, the baseline has 26.6% no-patch and 27.0% apply-failure rates; with SHERLOC findings these fall to 16.2% and 16.6%, respectively, paralleling the resolve-rate lift. For strong models, no-patch and apply-failure rates are already low, so the main measured effect remains search and token cost rather than patch

production. Table 16 shows the same pattern with the smaller Qwen3-30B localizer: localization-phase tokens decrease in every measured cell.

M. Token Cost Under the Masked-Description Control

Table 17 checks whether masking repository/file hints in the problem statement makes repair agents spend more tokens on localization. Across completed baseline-masked pairs, the effect is mostly small and mixed: with the exception of the Qwen3-Coder-480B-A35B/SWE-Agent outlier (+23.0%), localization-token deltas range from −12.0% to +10.9%. Thus, masking problem-statement identifiers generally does not force substantially more localization effort; agents largely follow similar exploration routines, or infer structure from the remaining context.

N. File-Level Localization: Baseline vs. SHERLOC

For each SWE-BENCH VERIFIED instance we extract the implicit localization output of baseline and masked agent runs, i.e., the set of *existing source files* that the agent actually modifies in its model patch, excluding agent-created reproduction, debug, or test scripts (any hunk introducing a new file is filtered out by checking for new file mode or `− /dev/null`). We compare these file sets to SHERLOC’s predicted files and to the gold patch files. Table 18 reports the full per-instance file-level localization breakdown that complements the compute and resolve-rate view in Table 15: Hit@1, macro precision/recall, and macro F1 over all 500 tasks. Instances where the agent never touches an existing source file contribute zero, so the metric jointly rewards correct file selection and the willingness to commit to one.

Reading. Table 15 reports compute and resolve rate; this table provides the matched file-level localization-metric breakdown. SHERLOC dominates baseline/masked agent localization for the weaker models, while MiniMax-M2.5 already localizes near saturation. Masked ($\approx$ baseline) values across rows indicate that masking file/repo identifiers in the problem statement is not a strong contamination control here: capable agents recover localization quality without those hints. Table 19 isolates the Qwen3-30B localizer used for the smaller-finding runs: it is less accurate than Qwen3-235B SHERLOC, but remains stronger than the baseline localization traces for the weaker repair-agent settings.

Table 15: **Localization-phase and full-run costs with and without SHERLOC findings.**

Per-instance means over 500 SWE-BENCH VERIFIED tasks; the localization phase is the trace prefix before the first source-edit action. The “**Total pipeline**” row adds the standalone-localizer cost (top of table) to the agent’s own interaction cost with the findings. Δ reports percentage change from baseline for cost columns and percentage-point change for resolve rate.

Setting	Loc. turns	Loc. input	Loc. output	Loc. total	Full-run total	Resolved
Standalone localizer						
SHERLOC (Qwen3-235B)	4.7	9.0k	19.5k	28.6k	–	–
Qwen3-Coder-30B-A3B / SWE-Agent						
Baseline	12.3	100.5k	1.3k	101.8k	622.4k	44.7%
Total pipeline	10.5	45.4k	20.4k	65.8k	433.6k	54.0%
Δ vs baseline	-14.6%	-54.8%	+1474.9%	-35.4%	-30.3%	+9.3 pp
Qwen3-Coder-30B-A3B / OpenHands						
Baseline	21.6	358.3k	2.8k	361.1k	1630.5k	49.4%
Total pipeline	17.7	212.7k	21.3k	234.1k	1299.4k	53.0%
Δ vs baseline	-18.1%	-40.6%	+658.7%	-35.2%	-20.3%	+3.6 pp
Qwen3-Coder-Next / SWE-Agent						
Baseline	53.3	1592.5k	5.6k	1598.1k	3025.9k	45.2%
Total pipeline	45.7	1034.9k	24.0k	1058.9k	2805.6k	53.2%
Δ vs baseline	-14.3%	-35.0%	+324.5%	-33.7%	-7.3%	+8.0 pp
Qwen3-Coder-Next / OpenHands						
Baseline	40.5	1140.8k	4.6k	1145.4k	3221.4k	44.6%
Total pipeline	33.0	703.9k	22.7k	726.6k	3267.8k	53.2%
Δ vs baseline	-18.5%	-38.3%	+394.5%	-36.6%	+1.4%	+8.6 pp
Qwen3-Next-80B-A3B-Instruct / SWE-Agent						
Baseline	9.2	160.0k	1.4k	161.4k	617.0k	38.8%
Total pipeline	8.6	41.4k	20.1k	61.5k	318.3k	49.4%
Δ vs baseline	-6.5%	-74.1%	+1373.0%	-61.9%	-48.4%	+10.6 pp
Qwen3-Next-80B-A3B-Instruct / OpenHands						
Baseline	9.6	179.6k	2.0k	181.6k	626.8k	38.6%
Total pipeline	10.9	112.4k	21.4k	133.8k	420.1k	50.4%
Δ vs baseline	+13.5%	-37.4%	+965.4%	-26.3%	-33.0%	+11.8 pp
MiniMax-M2.5 / SWE-Agent						
Baseline	11.3	114.3k	2.0k	116.3k	1556.3k	74.4%
Total pipeline	12.5	84.8k	20.8k	105.6k	1206.1k	70.4%
Δ vs baseline	+10.6%	-25.8%	+918.5%	-9.2%	-22.5%	-4.0 pp
MiniMax-M2.5 / OpenHands						
Baseline	22.1	423.7k	4.6k	428.3k	1591.3k	72.2%
Total pipeline	19.1	270.8k	22.6k	293.5k	1395.0k	67.2%
Δ vs baseline	-13.6%	-36.1%	+394.3%	-31.5%	-12.3%	-5.0 pp
Qwen3-Coder-480B-A35B / SWE-Agent						
Baseline	15.2	187.6k	1.3k	188.9k	1157.0k	63.0%
Total pipeline	10.8	44.5k	20.0k	64.6k	596.5k	63.0%
Δ vs baseline	-28.9%	-76.3%	+1408.8%	-65.8%	-48.4%	+0.0 pp
Qwen3-Coder-480B-A35B / OpenHands						
Baseline	24.5	470.0k	2.8k	472.8k	2067.5k	61.6%
Total pipeline	21.4	303.0k	21.6k	324.6k	1862.9k	62.6%
Δ vs baseline	-12.7%	-35.5%	+669.5%	-31.3%	-9.9%	+1.0 pp

Table 16: **Localization-phase efficiency with Qwen3-30B SHERLOC findings.** Per-instance means over 500 SWE-BENCH VERIFIED tasks for repair runs that receive findings produced by Qwen3-30B-A3B-Thinking-2507. “Qwen3-30B total” columns add the standalone Qwen3-30B SHERLOC localizer cost (7.2 turns; 10.9k input + 35.1k output = 46.0k total tokens) to the repair-agent run. Resolve rates are shown as baseline $\rightarrow$ with Qwen3-30B SHERLOC findings.

Model	Framework	Baseline loc. tok	Qwen3-30B loc. tok	Δ loc.	Baseline full tok	Qwen3-30B full tok	Δ full	Resolved
Qwen3-Coder-30B-A3B	SWE-Agent	101.8k	85.4k	-16.1%	622.4k	574.4k	-7.7%	44.7% $\rightarrow$ 43.8%
	OpenHands	361.1k	246.5k	-31.7%	1630.5k	1351.8k	-17.1%	49.4% $\rightarrow$ 42.2%
Qwen3-Coder-Next	SWE-Agent	1598.1k	1224.6k	-23.4%	3025.9k	2888.2k	-4.6%	45.2% $\rightarrow$ 49.2%
	OpenHands	1145.4k	840.5k	-26.6%	3221.4k	3197.9k	-0.7%	44.6% $\rightarrow$ 45.4%

Table 17: **Baseline vs. masked-description token cost.** “Loc.” is the localization phase (the prefix of the trajectory before the first source-edit action). Per-instance means; deltas are masked relative to baseline.

Model	Framework	Loc. turns	Loc. tokens	Δ loc. tok	Full-run tokens	Δ full tok	Resolved
Qwen3-Coder-30B-A3B	SWE-Agent	12.3 $\rightarrow$ 12.8	101.8k $\rightarrow$ 111.4k	+9.4%	622.4k $\rightarrow$ 997.4k	+60.2%	44.7% $\rightarrow$ 45.4%
	OpenHands	21.6 $\rightarrow$ 21.7	361.1k $\rightarrow$ 368.2k	+2.0%	1630.5k $\rightarrow$ 1644.9k	+0.9%	49.4% $\rightarrow$ 46.0%
Qwen3-Coder-Next	SWE-Agent	53.3 $\rightarrow$ 52.7	1598.1k $\rightarrow$ 1599.8k	+0.1%	3025.9k $\rightarrow$ 2871.4k	-5.1%	45.2% $\rightarrow$ 41.2%
	OpenHands	40.5 $\rightarrow$ 40.4	1145.4k $\rightarrow$ 1131.1k	-1.2%	3221.4k $\rightarrow$ 3125.6k	-3.0%	44.6% $\rightarrow$ 44.6%
Qwen3-Next-80B-A3B-Instruct	SWE-Agent	9.2 $\rightarrow$ 9.0	161.4k $\rightarrow$ 142.1k	-12.0%	617.0k $\rightarrow$ 607.1k	-1.6%	38.8% $\rightarrow$ 41.6%
	OpenHands	9.6 $\rightarrow$ 9.9	181.6k $\rightarrow$ 190.9k	+5.1%	626.8k $\rightarrow$ 697.9k	+11.3%	38.6% $\rightarrow$ 38.2%
MiniMax-M2.5	SWE-Agent	11.3 $\rightarrow$ 12.3	116.3k $\rightarrow$ 129.0k	+10.9%	1556.3k $\rightarrow$ 1642.7k	+5.6%	74.4% $\rightarrow$ 74.5%
	OpenHands	22.1 $\rightarrow$ 22.5	428.3k $\rightarrow$ 437.7k	+2.2%	1591.3k $\rightarrow$ 1586.6k	-0.3%	72.2% $\rightarrow$ 71.8%
Qwen3-Coder-480B-A35B	SWE-Agent	15.2 $\rightarrow$ 16.1	188.9k $\rightarrow$ 232.4k	+23.0%	1157.0k $\rightarrow$ 1269.0k	+9.7%	63.0% $\rightarrow$ 61.8%
	OpenHands	24.5 $\rightarrow$ 24.6	472.8k $\rightarrow$ 470.5k	-0.5%	2067.5k $\rightarrow$ 2066.1k	-0.1%	61.6% $\rightarrow$ 62.8%

Table 18: **File-level localization quality on SWE-Bench Verified.** SHERLOC’s predicted files vs. the existing-source files modified by baseline and masked agent runs; higher is better. **Evaluable (n)** is the number of instances where the method emits at least one file (SHERLOC predicts files for almost all instances; baseline/masked counts equal the rate at which the agent commits a real source edit). Hit@1, Recall, Precision, and F1 are per-instance means over all 500 instances, with non-evaluable instances contributing zero.

Model	Framework	Method	Hit@1	Recall	Precision	F1	Evaluable (n)
Qwen3-235B-A22B-Thinking-2507	SHERLOC		0.884	0.813	0.876	0.835	498 / 500
Qwen3-Coder-30B-A3B	SWE-Agent	Baseline	0.772	0.730	0.763	0.734	437 / 500
		Masked	0.788	0.735	0.781	0.743	457 / 500
	OpenHands	Baseline	0.752	0.759	0.748	0.730	461 / 500
		Masked	0.734	0.738	0.725	0.708	456 / 500
Qwen3-Coder-Next	SWE-Agent	Baseline	0.562	0.553	0.551	0.542	305 / 500
		Masked	0.494	0.483	0.485	0.474	271 / 500
	OpenHands	Baseline	0.562	0.584	0.543	0.547	357 / 500
		Masked	0.562	0.573	0.540	0.538	350 / 500
Qwen3-Next-80B-A3B-Instruct	SWE-Agent	Baseline	0.760	0.721	0.761	0.728	441 / 500
		Masked	0.766	0.726	0.766	0.731	447 / 500
	OpenHands	Baseline	0.752	0.776	0.752	0.738	480 / 500
		Masked	0.744	0.775	0.734	0.731	479 / 500
MiniMax-M2.5	SWE-Agent	Baseline	0.896	0.864	0.888	0.863	489 / 500
		Masked	0.886	0.848	0.880	0.851	461 / 500
	OpenHands	Baseline	0.824	0.846	0.818	0.809	493 / 500
		Masked	0.834	0.855	0.823	0.816	487 / 500
Qwen3-Coder-480B-A35B	SWE-Agent	Baseline	0.868	0.834	0.844	0.821	491 / 500
		Masked	0.862	0.833	0.839	0.818	488 / 500
	OpenHands	Baseline	0.816	0.837	0.773	0.776	490 / 500
		Masked	0.812	0.837	0.785	0.785	490 / 500

Table 19: **Standalone Qwen3-30B SHERLOC localization vs. baseline repair-agent traces.** The injected-finding repair runs in Table 16 do not receive a separate localization score; their findings are supplied by the standalone SHERLOC row.

Model	Framework	Method	Hit@1	Recall	Precision	F1	Evaluable (n)
Qwen3-30B-A3B-Thinking-2507	SHERLOC		0.786	0.751	0.785	0.750	497 / 500
Qwen3-Coder-30B-A3B	SWE-Agent	Baseline	0.772	0.730	0.763	0.734	437 / 500
	OpenHands	Baseline	0.752	0.759	0.748	0.730	461 / 500
Qwen3-Coder-Next	SWE-Agent	Baseline	0.562	0.553	0.551	0.542	305 / 500
	OpenHands	Baseline	0.562	0.584	0.543	0.547	357 / 500